Probing Reinforcement-Learned Representations for Geo-Free Guard Placement in the Vertex-Guard Art Gallery Problem

Domagoj Ševerdija[a,*], Jurica Maltar[a], Nathan Chappel, Domagoj Matijević[a]

[a]*School of Applied Mathematics and Informatics, University J. J. Strossmayer of Osijek, Croatia*

Abstract

Neural combinatorial optimization (NCO) has shown that policies trained by reinforcement can construct strong solutions to NP-hard problems directly from raw instances. What such a policy actually *learns*, as opposed to what its decoder *expresses*, remains much less clear. We study this distinction on the vertex-guard Art Gallery Problem, the NP-hard task of choosing polygon vertices from which to observe an entire region. A pointer-network policy is trained from a coverage-aware reward over its own rollouts under the constraint we call *geo-free* inference: at test time it sees only vertex coordinates, with no visibility computation and no geometric oracle. The constraint applies only to inference; during training and evaluation we use visibility freely, to compute the reward, build the probe's targets, and score coverage. The policy places guards economically, but a tail of under-covered polygons remains, and the tail grows on instances larger than any seen in training. To find the cause we freeze the encoder and read its embeddings with a small single-shot classifier, again geo-free at inference. The classifier removes most of the tail in and out of distribution, up to five times the training size, cutting under-covered polygons by about an order of magnitude at a reported cost in guard count. A capacity ladder (linear probe, attention-free MLP, full probe) and an encoder–seed ablation attribute this gain to the representation rather than to the probe's capacity. The reinforcement-trained representation therefore already encodes much of the geometry that coverage requires; on the better-supported of two readings, the residual failures are decoder calibration rather than missing representation. For this one policy, probing the frozen encoder is a practical way to measure what the solver learned.

Keywords: Art Gallery Problem, neural combinatorial optimization, reinforcement learning, pointer networks, representation probing

*Corresponding author

Email addresses: dseverdi@mathos.hr (Domagoj Ševerdija), jmaltar@mathos.hr (Jurica Maltar), nathan.s.chappel@gmail.com (Nathan Chappel), domagoj@mathos.hr (Domagoj Matijević)

1. Introduction

The Art Gallery Problem (AGP) asks for the minimum number of guards needed to observe every point of a simple polygon [1, 2], with applications in surveillance, sensor placement, and robotics [3, 4]. We focus on the *vertex-guard* variant, in which guards must be chosen from the polygon's vertices; the problem remains NP-hard and is a discrete instance of geometric set cover with non-uniform, polygon-dependent visibility regions. The question this paper investigates is whether a neural policy can learn to place such guards from coordinates alone, trained from a coverage-aware reward signal over its own rollouts, with no visibility matrix and no geometric oracle at the moment it chooses guards. We are not asking whether a learned solver can match a classical greedy heuristic on guard count — it cannot, and we do not claim it does. We are asking whether the underlying combinatorial-geometric task is learnable when the model is denied any explicit geometric input at test time.

Why geo-free inference. One could object that the coordinates are given, so a solver could just compute visibility from them; a visibility-based greedy does exactly that, at a lower guard count. Geo-free inference withholds the computation, not the information. The geo-free policy and classical greedy start from the same coordinates: greedy evaluates visibility at every step, while the geo-free model must have learned it. This makes the constraint a measurement tool. A solver that may query a visibility oracle at inference can reproduce greedy or local search, so its learned behaviour cannot be told apart from what the oracle computes (Section 2.3). Withholding the oracle removes that confound, and it is a fair restriction: it denies the model a computation, not information.

Geo-free inference also shifts where the visibility cost is paid. In a classical pipeline that cost is the largest per-instance expense — a visibility polygon per vertex, then a polygon-union coverage per candidate set — and it recurs for every polygon. A geo-free policy pays it once, at training, and then answers each instance with a single coordinate-only forward pass in tens of milliseconds on a GPU, with model memory independent of n (Sections 5.3 and 6.5). This trade of a high fixed cost for a low marginal one helps when many instances must be solved or latency is tight. That the trade is even available is not an assumption: the probing study is what shows the required visibility structure can be learned from coordinates. Several concrete settings fit. In design-space search, a layout-optimization loop scores many candidate polygons and needs a fast coverage estimate to guide it. In triage, a one-pass covering set marks the easy polygons and warm-starts an exact vertex-guard integer program [5] on the rest. In interactive tools, coverage feedback must keep pace with edits, which exact visibility at n in the thousands cannot. In each case the learned model is a fast first pass and an exact method remains the authority on quality, so the probe's over-guarding (Section 6) is acceptable. We do not claim it replaces a

visibility-based solver on guard count, or that it helps where an exact oracle is cheap.

The reinforcement method we evaluate is an LSTM pointer-network policy trained with Preference Optimization (PO) [6] on Bradley–Terry preference pairs [7] drawn from the policy's own rollouts. The policy emits a vertex-index sequence with an EOS token; the reward couples coverage and guard usage. PO replaces REINFORCE advantages [8, 9] with binary preferences between pairs of policy rollouts, preserving a usable gradient signal even when the reward saturates under the coverage constraint [6]. We refer to inference under this policy as *geo-free* — the network reads only vertex coordinates at the moment it places guards, never querying a visibility oracle (defined precisely in Section 2.3).

The policy works, to a degree. On a held-out in-distribution split its guard sets are competitive in cardinality with classical baselines, but the per-polygon coverage distribution has a tail: a non-trivial fraction of polygons end up below the 0.95 coverage gate one would want a deployed solver to clear, and the failure rate grows on out-of-distribution polygons whose vertex counts exceed the training range. The reinforcement method, by itself, offers no coverage guarantee.

To check whether this residual reflects a true limit of the reinforcement-learned representation or only a calibration problem in the decoder, we train a small per-vertex classifier conditioned on the policy's frozen encoder embeddings — together with the vertex coordinates and a binary indicator of the policy's own guard set, all geo-free — and predict a vertex-inclusion probability, controlled by a single scalar threshold. The classifier has no access to visibility information at inference, and it compresses the coverage tail substantially both in and out of distribution. A no-encoder ablation that zeroes the embeddings while keeping the coordinates and seed indicator (Section 6.2), together with a probe-capacity ladder that holds the encoder fixed while shrinking the probe, isolates the encoder's contribution from the other inputs and from the probe's own capacity; we read the gap as evidence that the reinforcement-trained encoder already carries geometric structure a small probe can turn into a gate-clearing decision — so the policy's coverage tail reflects, on the better-supported reading, what its decoder expressed rather than a limit of the representation. The cost of clearing the tail is a controlled rise in guard count, which we report explicitly.

Contributions. This paper makes four contributions.

C1 A representation–decoder decomposition for neural combinatorial optimization. We adapt the probing methodology of representation analysis [10, 11, 12] to a reinforcement-trained combinatorial solver: freezing the policy's encoder and training a single-shot per-vertex probe over its embeddings (with the vertex coordinates and the policy's seed) isolates what the learner *represents* from what its decoder *expresses*, with a 2×2 encoder–seed ablation and a probe-capacity ladder (linear, attention-free MLP, full probe) as controls. Concurrent work has begun probing neural

routing solvers [13, 14]; to our knowledge ours is the first probing study of an RL-trained policy on a *constrained geometric covering* problem, and the first to read a coverage decision (per-vertex guard membership) from a frozen encoder under a geo-free inference constraint.

C2 Out-of-distribution generalization at scale. On a large held-out set spanning the training size range and extending to roughly five times beyond it, the probe raises the average fraction clearing the 0.95 coverage gate across four probe seeds from 85% (policy seed) to 99%; the gain holds on the genuinely larger-than-training polygons alone (85% $\rightarrow$ 98.7%, Section 6.3).

C3 Geo-free inference as a protocol. Restricting test-time inputs to vertex coordinates and learned features derived from them — never a visibility oracle — isolates what a learner internalizes from what such an oracle could compute for it, making "no explicit geometric knowledge" a measurable property rather than a rhetorical one (Section 2.3).

C4 A structural finding on post-processing. Learned iterative editors fail to improve the policy's seed without dropping coverage — due to error accumulation, stop-time miscalibration, and train/inference drift — whereas the single-shot probe avoids all three by design and is empirically a fixed point: re-running it on its own output changes nothing (Section 6.4). We read this as support for the single-shot design as a stable reading of the encoder; the fixed point is an empirical property, not a proof that the probe is a clean representation readout.

We do not claim to beat classical greedy or local-search heuristics on guard count; we claim, and document, that vertex-guard placement is learnable to a measurable degree by a reinforcement method that never reads an explicit visibility object at inference. The components we use, pointer networks, Bradley–Terry preference optimization, frozen-representation probing, and a Transformer per-vertex classifier, are all established. Our contribution is their combination and adaptation to a constrained geometric covering problem, and the representation–decoder measurement it enables.

The remainder of the paper is organized as follows. Section 2 establishes notations and definitions. In Section 3 related work is reviewed. Section 4 describes the reinforcement method and the representation probe. Finally, Section 5 describes the experimental setup, Section 6 reports the results, followed by discussion in Section 7, limitations in Section 8, and the conclusion in Section 9.

2. Problem and Notation

2.1. Polygons and visibility

Let $P \subset \mathbb{R}^2$ denote a simple polygonal region (the closed set consisting of its interior and boundary), with n vertices at coordinates $x_1, \ldots, x_n \in \mathbb{R}^2$ and vertex index set $V(P) = \{1, \ldots, n\}$. For two points $p, q \in P$ we say that p *sees*

q when the closed segment between them stays inside the polygon, $\overline{pq} \subseteq P$. Guards are placed at vertices: for a vertex-guard set $S \subseteq V(P)$, the visibility region and area-normalized coverage are

$$\mathrm{Vis}_P(S) \;=\; \{\, q \in P : \exists\, i \in S,\; x_i \text{ sees } q \,\}, \qquad \mathrm{Cov}_P(S) \;=\; \frac{\mathrm{Area}(\mathrm{Vis}_P(S))}{\mathrm{Area}(P)} \in [0,1]. \tag{1}$$

2.2. The vertex-guard variant

The *vertex-guard Art Gallery Problem (AGPVG)* asks for the smallest vertex-guard set that covers the whole polygon. We define the *vertex-guard optimum* directly:

$$\mathrm{OPT}(P) \;=\; \min\big\{|S| : S \subseteq V(P),\; \mathrm{Cov}_P(S) = 1\big\}. \tag{2}$$

Restricting guards to $V(P)$ reduces placement to a finite discrete choice over n vertices — the natural action space for a pointer-network policy [15, 9, 16, 17] — while retaining the full NP-hardness and non-uniform visibility structure of the problem.

Note that equation (2) is a *constrained* single-objective problem — minimize the guard count subject to the hard constraint $\mathrm{Cov}_P(S) = 1$ — and is a discrete set-cover instance, where coverage is the submodular union of per-guard visibility regions. This is what separates it from the routing problems on which neural combinatorial solvers are usually studied, where any permutation is feasible and only a single scalar cost is minimized. A learner denied a visibility oracle at inference (the geo-free constraint, Section 2.3) cannot certify the coverage constraint while placing guards; it must instead trade coverage against guard count, so the single constrained objective becomes a two-axis operating curve.

2.3. Geo-free inference

We will say that an inference procedure is *geo-free* if its inputs at test time are limited to the polygon's vertex coordinates and learned features derived from them — no polygon visibility matrix, no CGAL-computed visibility regions, no per-vertex area or visibility-fraction feature. Geo-free inference does not preclude using exact visibility during evaluation (to report coverage after the fact) or during training (to compute rewards, gate trajectories, or build supervised targets); it restricts what the model reads at the moment it places guards. A solver allowed to query a visibility oracle at every decision step — as classical local search and active-search decoding both do — can in principle simulate greedy or local search, and a measurement of what such a solver has learned would be conflated with a measurement of what its oracle can compute. The geo-free constraint isolates the first question from the second, which is what makes "no explicit geometric knowledge" a measurable property.

3. Related Work

This section places the present work in four lines of research: algorithms for the Art Gallery Problem (classical and learned), neural combinatorial optimization with reinforcement learning, supervised post-processing of learned solvers, and probing of learned representations.

The Art Gallery Problem. Lee and Lin [1] established NP-hardness for several variants of the AGP; O'Rourke [2] surveys the foundational geometric and combinatorial results. Work on the problem has traditionally followed three lines. Exact algorithms solve smaller instances to optimality, most effectively via integer programming: Couto et al. [5] compute exact vertex-guard optima this way, and we treat their published instance library [18], with its exact optima, as the source of polygons in our experiments. Approximation algorithms provide provable guarantees — Ghosh [19] gives a logarithmic-factor method for guarding simple polygons. Heuristics without theoretical guarantees, of which the classical greedy rule [20] that places guards to maximize marginal coverage is the most common, are reliable on small to mid-sized polygons. Because area coverage is a monotone submodular function of the guard set, this marginal-coverage rule is exactly the greedy submodular-maximization heuristic and inherits its diminishing-returns approximation guarantee [21]; the guarantee is available only to a solver that can evaluate marginal coverage while it selects, which a geo-free learner cannot (Section 2.3), so we forgo such guarantees by construction — the price of measuring what the representation has learned rather than what an oracle computes. The same exact/approximation/heuristic split recurs across the problem's many variants — for instance, guarding 1.5-dimensional terrains admits constant-factor approximations [22, 23]. Local-search refinement of a candidate guard set by swapping or removing vertices is the standard high-quality post-processing step. We use classical greedy purely as a non-learned evaluation reference, not in competition with the reinforcement method on guard count.

Reinforcement learning has been applied to the AGP and to closely related coverage problems before, but in settings quite different from ours. Liao et al. [24] discretize the polygon into a grid and train a per-instance tabular Q-learning agent that trades guard count against covered area; Kaba et al. [25] cast the 3D view-planning problem — a sensor-coverage set-cover — as an MDP solved with value-based RL (SARSA, Q-learning, temporal-difference) guided by a geometry-aware score, outperforming the greedy baseline. Both methods consult coverage or visibility while placing guards and learn a fresh policy for each instance. We differ on three axes: the action space is the polygon's own vertices rather than grid cells or candidate viewpoints; a single amortized policy generalizes to unseen polygons without retraining; and inference is geo-free, with no visibility computation at the moment guards are placed.

Reinforcement learning for combinatorial optimization. Pointer networks [15] introduced the architecture of attending over input tokens to produce variable-

length output index sequences, and were extended to combinatorial optimization with reinforcement learning by Bello et al. [9] using REINFORCE [8] with a learned baseline; Deudon et al. [26] similarly train TSP construction heuristics by policy gradient. Subsequent work introduced attention-only architectures for routing problems [16], policy-optimization variants such as POMO [17] that exploit problem symmetries, and Transformer pointer policies such as Pointerformer [27] aimed at generalizing to large instances — a concern we share, since our evaluation stresses out-of-distribution polygon sizes. We adopt the more recent preference-optimization (PO) recipe of Pan et al. [6], which trains the policy with a Bradley–Terry [7] ranking loss over pairs of solutions sampled from the policy itself, replacing the variance-heavy REINFORCE objective. We treat PO as a reinforcement-style procedure: the loss is computed from policy rollouts on the environment (the polygon), the gradient direction is determined by the reward (coverage versus guard usage), and no supervised labels for the action sequence are used. A broader methodological survey is provided by Bengio et al. [28].

A recurring feature of this literature is worth making explicit, because AGP departs from it. Routing problems such as the TSP carry a single scalar objective and free feasibility — any permutation is a valid tour — and even constrained variants such as capacitated routing usually enforce feasibility by masking infeasible actions at decode time, which presumes an oracle for the constraint. Vertex-guard AGP has neither property: it is a constrained set-cover problem (Section 2.2), and under geo-free inference the coverage constraint cannot be masked in, so it can only be learned and traded off against guard count — the operating curve our threshold sweep traces. The closest learned analogues are RL methods that approximate Pareto frontiers for bi-objective routing [29, 30]; but those balance two *soft* costs, whereas one of our two axes is a hard feasibility constraint, making PA-Net's Lagrangian relaxation of a constraint [29] a closer match than a symmetric Pareto front. On the optimization side, Caramanis et al. [31] prove that policy-gradient solution-samplers enjoy benign landscapes for a broad class of problems (Max-/Min-Cut, Max-k-CSP, bipartite matching, TSP), but their guarantee assumes a single cost that is *bilinear* in instance and solution features and imposes no hard feasibility constraint. AGP's guard-count term is linear and would fit, but its coverage term is the submodular union of visibility regions and its full-coverage requirement is a hard, geo-free-uncertifiable constraint — placing AGP outside their analyzed class and helping explain why a naive policy gradient struggles here (Section 4.1).

Supervised post-processing of learned solvers. An orthogonal line of work post-processes a base solver's output with a second model trained by supervised or imitation learning. For combinatorial problems with hard feasibility constraints (such as covering), iterative supervised editors must contend with compounding errors and a train–inference distribution gap; DAgger [32] and its variants partially address the latter. We use a single-shot supervised post-processor (the SetPredictor) not as a competing solver but as a *probe* of the RL-trained encoder's representation. The framing is methodological: if a small classifier

reading frozen RL embeddings can recover the coverage the decoder left short, the RL encoder must already carry the relevant geometric information. Directly cloning search solutions with a recurrent decoder, by contrast, fails on this problem for a structural reason — the teacher-forcing cascade we return to in Section 6.4 — which is why we read the frozen encoder with a single-shot, recurrence-free probe rather than a sequential supervised decoder.

Probing learned representations. The methodology of freezing a trained network and reading its internal representations with a small supervised classifier originates in the analysis of vision and language models: linear classifier probes diagnose what intermediate layers encode [10], control tasks calibrate how much of a probe's accuracy reflects the representation rather than the probe's own capacity [11], and Belinkov [12] surveys the family. A central methodological caveat runs through this literature: a sufficiently expressive probe can achieve high accuracy by *computing* the target itself rather than reading it off the representation, so probing results must be interpreted against a capacity control [11]. We take this concern seriously below by reporting a capacity ladder (a zero-hidden-layer linear probe, an attention-free MLP, and the full SetPredictor) alongside a no-encoder control task, so that the effect we attribute to the representation is the part that survives holding probe capacity fixed and removing the encoder.

Probing has only recently reached neural combinatorial optimization, and the closest work is concurrent. Zhang et al. [13, 33] freeze trained routing models (AM, POMO, LEHD) on the TSP and CVRP and read their embeddings with *linear* probes — for a low-level geometric quantity (pairwise Euclidean distance), for solution-structure signals (whether a link avoids a myopic greedy choice; whether two nodes lie on the same route), and for a capacity-constraint signal — with a coefficient-significance tool (CS-Probing) that localizes which embedding dimensions carry each. Narad et al. [14] similarly probe TSP embeddings for node-removal and edge-forbid sensitivity. We use the same freeze-and-probe method but probe something different in a different setting. Our problem is covering, not routing: under geo-free inference the coverage constraint cannot be enforced by decode-time masking, so the model must learn it and trade it against guard count. Our target is global, per-vertex membership in a guard set whose visibility regions must cover the polygon, rather than a pairwise quantity like inter-node distance; visibility depends on the whole boundary, so recovering it from the embedding says more about learned geometry than recovering a distance does. The geo-free limit on what the probe may read at inference has no counterpart in the routing studies. And where they probe linearly, avoiding the capacity question but seeing only linear signal, our probe is an expressive Transformer that also serves as the post-processor, so we run a capacity ladder (linear, attention-free MLP, full Transformer; Section 6.2) to confirm the signal comes from the representation and not the probe. The learning frameworks differ only in recipe — our Bradley–Terry preference optimization against their policy-gradient (AM, POMO) or supervised (LEHD) training — and since both lines include RL models we rest no claim on it. Our no-encoder ablation is the

control task [11]: it holds probe capacity and training fixed while removing the representation, so the gap is attributable to the encoder.

4. Method

We describe the reinforcement method (Section 4.1) and the single-shot representation probe (Section 4.2). The empirical diagnostic that most motivates the single-shot design, i.e., why iterative editors fail to improve the seed, is deferred to the results (Section 6.4).

4.1. The reinforcement method: PO/BT pointer policy

Architecture. The policy is a compact pointer network with a single-layer, unidirectional LSTM [34] encoder and an attention-based decoder. A polygon P with vertex coordinates $(x_1, \ldots, x_n)$ is consumed coordinate-wise (no visibility input). Running the LSTM over the vertex sequence yields per-vertex hidden states that serve as the embeddings $(h_1, \ldots, h_n)$, $h_i \in \mathbb{R}^d$, where d is the encoder embedding dimension (its value, and those of all dimensions introduced below, is set in Section 5.2). The decoder attends over the encoder at each step, emits a vertex index or an *end-of-sequence* (EOS) token, and stops at EOS or when the maximum length is reached. The decoder outputs a sequence $\pi = (\pi_1, \ldots, \pi_{|\pi|})$ while the resulting guard set is $S(\pi) = \{\pi_t : \pi_t \neq \text{EOS}\}$. All metrics, i.e., coverage, $|S|/n$, $|S|/\text{OPT}$, are computed from $S(\pi)$. We denote the joint encoder–decoder distribution by $p_\theta(\pi \mid P)$, where θ collects all policy parameters.

The encoder has no training signal of its own: its weights change only because the reward gradient back-propagates from the decoder's rollouts. If the encoder already holds the geometric information needed for feasibility, a small classifier reading its frozen embeddings should be able to recover the guard set the decoder failed to express. This is why we probe the encoder inside the trained policy by freezing it and attaching a small classifier. By contrast, a standalone model, i.e., an encoder + a probe trained from scratch, would reflect its own learning, not what the reinforcement policy internalized. The policy's greedy decode additionally supplies the seed guard set that the representation probe (Section 4.2) later refines.

Why an autoregressive pointer. Vertex-guard placement is a set-cover problem: whether a vertex should be a guard depends on what the guards already chosen leave uncovered. An autoregressive pointer captures this dependency naturally by conditioning each pick on the partial solution (Eq. (5)), and it decides cardinality through the EOS token rather than a fixed threshold. A model scoring vertices independently would miss this structure — which is also why the probe (Section 4.2) is not a standalone solver: unable to be autoregressive, it takes the policy's greedy seed as an input instead.

The reason the decoder is necessary once the probe works, i.e., why we cannot discard it, is that the encoder's representation was shaped entirely by the reward

gradient flowing through the decoder, and the probe reads the decoder's seed as a feature. Removing the decoder removes both the representation and the probe's input. What we compare is encoder-and-decoder against encoder-and-probe, not the decoder against no-decoder. Reading the frozen encoder with a probe also recovers coverage on polygons far larger than those seen in training (Section 6.3), which autoregressive pointer policies alone cannot do [15, 27].

Reward and rollouts. For each polygon P we sample R rollouts from the policy and score each rollout's guard set $S(\pi)$ by a scalar utility that gates coverage at τ and then prefers smaller guard sets,

$$u(\pi \mid P) \;=\; \min\big(\mathrm{Cov}_P(S(\pi)),\, \tau\big) \;-\; \lambda\,\frac{|S(\pi)|}{|V(P)|} \;-\; \rho\,\max\big(0,\, \tau - \mathrm{Cov}_P(S(\pi))\big), \tag{3}$$

where $\tau = 0.99$ is the coverage gate, $\lambda = 1.0$ weights guard usage, and $\rho = 3.0$ penalizes coverage below the gate (Section 5.2). Capping the coverage term at τ removes any incentive to over-cover, so once a rollout meets the coverage gate the utility is driven purely by cardinality. Coverage during training is computed with the discretized-visibility sampler described next ($M = 500$ interior points per polygon), a cheap approximation of the exact coverage area.

Discretized-visibility coverage. Computing the exact coverage of a guard set — the area of the union of its vertices' visibility polygons — is too costly to repeat for the many rollouts and local-search candidates scored per polygon, so during training we estimate it by Monte-Carlo sampling. For each polygon we draw $M = 500$ points uniformly from its interior (rejection sampling within the bounding box, under a fixed per-polygon seed) and, for each vertex i, compute its *exact* visibility polygon $\mathrm{Vis}_P(\{i\})$ once with CGAL triangular-expansion visibility [35]. Recording which samples fall inside each visibility polygon yields a boolean visibility matrix $B \in \{0,1\}^{|V(P)|\times M}$ with $B_{is} = 1$ iff sample s is visible from vertex i, and the coverage of a guard set S is estimated by the covered-sample fraction

$$\widehat{\mathrm{Cov}_P(S)} \;=\; \frac{1}{M}\,\Big|\big\{\, s : \exists\, i \in S,\; B_{is} = 1 \,\big\}\Big|, \tag{4}$$

a uniform Monte-Carlo estimate of the area ratio $\mathrm{Cov}_P(S)$ of Eq. (1), evaluated as a bitwise OR over the rows of B. Because B depends only on the polygon, it is computed once and cached, so every subsequent coverage query during a rollout or a local-search edit costs an $O(|S|\,M)$ bitwise operation rather than a polygon-union computation. The per-vertex visibility polygons are exact; the only approximation is replacing the area integral by a sample average, whose standard error decays as $O(1/\sqrt{M})$. Exact coverage — the area of the union, integrated rather than sampled — is reserved for evaluation (Section 5.3).

Preference-optimization loss. We score a rollout by its *length-normalized* log-likelihood — the mean per-token log-probability under the factorization

$$p_\theta(\pi \mid P) = \prod_t p_\theta(\pi_t \mid \pi_{<t}, P)$$

Training objective	Mean cov	Mean $\lvert S\rvert$/OPT
REINFORCE, learned-critic baseline	0.904	3.49
REINFORCE, greedy rollout baseline	1.000	7.00
PO / Bradley–Terry (ours)	**0.978**	**1.30**

Table 1: Training-objective comparison on the combined `dev+test` pool (greedy decode). Coverage is geometric polygon coverage; $|S|$/OPT is the mean approximation ratio. The two REINFORCE rows are short 30-epoch runs scored under an *earlier* evaluation protocol: computed on the combined `dev+test` pool *before* the train/eval-leakage deduplication that produced the clean splits used in all other tables (Section 5.1), and with far fewer training epochs than the 200-epoch PO/BT policy. They are reported only as evidence for the *choice of training objective* — REINFORCE over-guards by 3.5–7× the optimum, a gap far too large to be a protocol artifact — and their absolute numbers are *not* matched cell-for-cell to the headline tables. The PO/BT entry is shown under the same earlier protocol for reference (which is why its $|S|$/OPT = 1.30 here differs from the clean-split `test` value in Table 3); the policy it refers to is the one used throughout the paper.

,

$$\bar{\ell}_\theta(\pi \mid P) \;=\; \frac{1}{|\pi|}\sum_{t=1}^{|\pi|} \log p_\theta(\pi_t \mid \pi_{<t}, P), \tag{5}$$

which keeps the preference from being biased by guard-set size, since AGP solutions have variable length (cf. Pan et al. [6]). For each pair (π^+, π^-) of rollouts with $u(\pi^+ \mid P) > u(\pi^- \mid P)$ above a coverage gate, we apply the Bradley–Terry log-likelihood

$$\mathcal{L}_{\mathrm{BT}}(\theta) \;=\; -\mathbb{E}_{(P,\pi^+,\pi^-)}\left[\log \sigma\big(\alpha\,[\,\bar{\ell}_\theta(\pi^+ \mid P) - \bar{\ell}_\theta(\pi^- \mid P)\,]\big)\right], \tag{6}$$

where $\alpha > 0$ is a temperature scaling the preference margin ($\alpha = 0.05$; Section 5.2). The gradient direction is driven purely by which of the two rollouts achieved a higher reward; no supervised target sequence is used.

The choice of PO over REINFORCE is deliberate. Once a rollout meets the coverage constraint, the reward of Eq. (3) flattens and the policy-gradient signal on guard count vanishes — a saturation effect specific to hard coverage constraints, not covered by benign-landscape guarantees for unconstrained problems [31]. We observe this failure directly: a value-critic REINFORCE policy over-guards by roughly 3.5× the optimum and a greedy-baseline variant by about 7× (Table 1). Bradley–Terry preferences between rollout pairs remain informative even when both rollouts saturate, since the signal depends on which rollout is relatively better, not on the absolute reward level. PO/BT reaches comparable coverage at 1.3× the optimum. At the end of training the encoder is frozen and treated as a fixed feature extractor for the rest of the paper; Figure 1 shows the late-phase policy holding a stable operating point — good coverage at low guard cost — rather than collapsing onto a single axis.

4.2. Probing the representation: a single-shot per-vertex classifier

The policy learns to decrease the cardinality but leaves a coverage tail (Section 6.1). We first verified that this tail is *reachable*: running a local-search

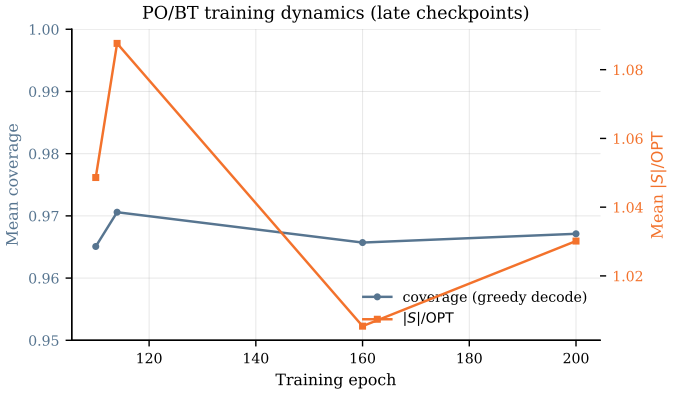

Figure 1: PO/BT training dynamics over the four late checkpoints (epochs 110, 114, 160, 200), evaluated on a fixed held-out sample of 100 polygons; per-epoch logging was not preserved, so early training is not shown (Section 8). Mean per-polygon coverage (left axis, zoomed to $[0.95, 1.0]$) and $|S|/\mathrm{OPT}$ (right axis) of the greedy-decoded policy stay within a narrow band with no sign of collapse; the movement is non-monotone rather than a clean trend, which is all four checkpoints can establish.

editor (add/remove/swap) on the policy's own greedy seed lifts every below-gate in-distribution polygon back over the 0.95 gate while *reducing* the guard count (the *LS on policy seed* row of Table 3). We treat this as a sanity check that settles the diagnosis — the missing coverage is a handful of local edits away from the seed, so it reflects decoder calibration rather than absent geometry — and that, as a by-product, yields a concrete target set $S^{\star}_{\mathrm{LS}}$. The open question is then whether that refinement can be *learned* from the frozen encoder, geo-free. A learned *iterative* editor over the seed cannot: it does not close the tail in any coverage-preserving regime (Section 6.4), because error accumulation, stop-time miscalibration, and train/inference drift make a rollout-based post-processor unreliable here, contrary to fine-tuning arguments of the policy in [6]. We therefore probe what the policy has learned with a *single-shot, rollout-free* classifier that recovers the coverage decision the decoder failed to make. Its inputs are geo-free — the frozen encoder embedding, the vertex coordinates, and the policy's seed indicator — with no visibility object at inference. Having no rollout and no stop decision, a single pass has nothing to drift and nothing to miscalibrate. We call it the SetPredictor and present it primarily as a representation diagnostic; the threshold sweep in Section 6 makes the diagnostic concrete, but the operating-curve framing is a side-effect of the measurement, not a deployment recipe.

For polygon P with vertex coordinates $x_1, \ldots, x_n$, the input feature for vertex i is

$$z_i \;=\; [\, h_i \,;\, x_i \,;\, \mathbf{1}\{i \in S(\pi_{\mathrm{seed}})\}\,] \;\in\; \mathbb{R}^{d+3}, \tag{7}$$

that is, the *frozen* d-dimensional encoder embedding h_i from the PO/BT policy, the 2D coordinates x_i, and a single binary indicator for whether vertex i is

selected by the policy's greedy-decoded seed π_{seed}.

The architecture is a small Transformer encoder (parameters ϕ) over the n vertex tokens with a sigmoid output head per token (sizes given in Section 5.2). The features z_i are linearly projected to width d and passed through L pre-norm self-attention blocks — each with H-head self-attention at width d, a GELU feed-forward sublayer of expansion factor f, and residual connections — followed by a final layer normalization; we write v_i for the resulting per-vertex hidden state. This v_i is the SetPredictor's *own* encoding of vertex i, and is distinct from the frozen policy embedding h_i, which enters only as part of the input feature z_i in Eq. (7). From the $(v_1, \ldots, v_n)$ we build two context vectors by mean-pooling: a *seed* context $c_{\text{seed}} = \frac{1}{|S(\pi_{\text{seed}})|} \sum_{i \in S(\pi_{\text{seed}})} v_i$, averaged over the vertices the policy placed in its seed, and a *global* context $c_{\text{glob}} = \frac{1}{n} \sum_{i=1}^{n} v_i$, averaged over all vertices of the polygon. Both are single vectors shared across vertices. Each per-vertex prediction concatenates the vertex's own v_i with these two context vectors and passes the result through a 2-layer GELU MLP head $(3d \to d \to 1)$:

$$\hat{p}_i \;=\; \sigma\big(\mathrm{MLP}([\, v_i \,\|\, c_{\text{seed}} \,\|\, c_{\text{glob}} \,])\big), \qquad i = 1, \ldots, n. \tag{8}$$

The output $\hat{p}_i \in [0, 1]$ is interpreted as the probability that vertex i should be in the final guard set. Thresholding at $t \in (0, 1)$ yields the guard set

$$S(t) \;=\; \{\, i \in \{1, \ldots, n\} : \hat{p}_i \geq t \,\}. \tag{9}$$

Objective. The probe is trained by supervised learning to predict membership in an LS-improved target set $S^{\star}_{\text{LS}}(P)$ — the policy's seed refined by local search, constructed as described in Section 5.2 — so it is not itself a reinforcement learner. We supervise on this refinement rather than on the exact optimum for two reasons: guard membership at the optimum is non-unique — the constraint in Eq. (2) fixes cardinality, not which vertices are chosen (Section 6.4) — so an optimal set is an ill-posed per-vertex label, and the optimum is in any case unavailable at scale (Section 5.3); the LS endpoint, by contrast, is a well-defined, coverage-restoring refinement of the policy's *own* seed. With $y_i = \mathbf{1}\{i \in S^{\star}_{\text{LS}}(P)\}$, a positive-class weight w_+ that balances the empirical non-guard-to-guard ratio, and $\mathcal{B}$ a mini-batch of polygons, the per-vertex weighted binary cross-entropy is

$$\mathcal{L}_{\text{BCE}}(\phi) \;=\; -\frac{1}{\sum_{P \in \mathcal{B}} |V(P)|} \sum_{P \in \mathcal{B}} \sum_{i=1}^{|V(P)|} \big[\, w_+ y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \,\big]. \tag{10}$$

Whether closing that tail is the encoder's doing rather than the probe's own capacity is settled not here but by the controls of Section 6.2: the no-encoder ablation and a zero-capacity linear probe.

Inference and the threshold knob. At inference we run the policy encoder once on the coordinates, greedy-decode the seed π_{seed}, form the features z_i of Eq. (7), run

one SetPredictor forward pass to obtain $(\hat{p}_1, \ldots, \hat{p}_n)$, and threshold to return $S(t)$ (Eq. (9)). The policy seed, the local-search editor, and the discretized-visibility scoring are all *training-time* scaffolding used only to build the labels $S^{\star}_{\mathrm{LS}}$; at inference the method is this single geo-free forward pass — no seed-refinement loop, no local search, no visibility object — so LS is part of how the probe is *trained*. The threshold t is the single knob trading coverage for guard count, swept over $t \in \{0.20, 0.25, 0.30\}$ in Section 6.2. Re-running the probe with its own output as the updated seed indicator changes nothing — it is empirically a fixed point after the first pass (Section 6.4) — so we use a single pass throughout.

5. Experiments

5.1. Dataset and partition

We draw polygons primarily from the *Art Gallery Problem with Vertex Guards* (AGPVG) instance library of Couto, de Rezende, and de Souza [18, 5], which collects several polygon families — random orthogonal (including large-area `fat` and small-area `min` variants), random simple (`randsimple`), random von Koch (`randvon`), complete von Koch (`vonkoch`), and boundary-simple (`simple`) — over vertex counts n ranging from 8 to 2,500. The library distributes vertex-guard optimal solutions for its instances, which we use directly as OPT.

We supplement the library with 313 additional random simple polygons (`random` class, $n \in \{20, 40, \ldots, 600\}$, 30 instances per size) to increase training density for random simple polygons in the in-distribution range; these share the same rational-coordinate format, and their vertex-guard optima were computed by solving the vertex-guard set-cover integer program. The splits we work with are listed in Table 2.[1] The held-out evaluation splits are deduplicated against `train` to remove any instance shared with it (train/eval leakage); every evaluation number in this paper is computed on these leak-free splits, and it is this deduplication that separates them from the earlier protocol under which the REINFORCE rows of Table 1 were scored.

5.2. Training setup

The PO/BT policy is the released `lstm_bt` checkpoint: a *single* PO/BT training run (seed 1234, 200 epochs), from which we keep the checkpoint with the best greedy-decoded `train` performance. It uses $R = 8$ rollouts per polygon, Bradley–Terry temperature $\alpha = 0.05$, and reward weights $\lambda = 1.0$ and $\rho = 3.0$ at the gate $\tau = 0.99$ (Eq. (3)); all PO coverage rewards are computed from discrete-visibility sampling ($M = 500$), after which the encoder is frozen. During development we compared several training *recipes* — REINFORCE baselines

[1] The 70/30 `dev`/`test` partition uses a seeded random shuffle (seed 1234) rather than an alphabetical-name split due to instance naming

Split	# polygons	n range	Use
train	8,867	8–198	Pointer + SetPredictor training
dev	840	16–198	SetPredictor checkpoint selection
test	362	8–192	Held-out evaluation (in-distribution)
ood	2,081	8–1,000	Out-of-distribution evaluation
ood-large	285	600–2,250	Extreme-OOD evaluation

Table 2: Dataset partition. The `dev` and `test` splits share the same source library files, partitioned 70/30 by seeded random shuffle (seed 1234): `dev` is used for checkpoint selection and `test` is the in-distribution held-out evaluation. The `ood` split is a larger held-out set spanning the training size range and extending well beyond it, to $n = 1000$ (about five times the training maximum); its genuinely larger-than-training polygons ($n > 198$) are analyzed separately in Section 6.3. The `ood-large` split is used for an extreme-OOD evaluation (Table 10).

(Table 1) and alternative fine-tuning objectives — and selected PO/BT among them by `train`-set performance; the released policy is one run of that chosen recipe, not the best of many random seeds (see Limitations, Section 8).

Inputs: a polygon's vertices are consumed by the LSTM in the source-library file order — no canonical start vertex and no orientation (clockwise/counterclockwise) are imposed — and the coordinates are min–max normalized per polygon to $[0, 1]^2$ before the encoder. The normalization makes the policy invariant to translation and to axis-aligned rescaling by construction; it is *not* invariant to rotation or to a cyclic re-indexing of the vertices, whose empirical effect we quantify in Section 8.

Targets: for each training polygon we obtain $S^{\star}_{\mathrm{LS}}(P)$ by running local search on the policy's seed under a coverage gate $\tau = 0.99$ against a discrete-visibility reference (Section 4.1); exact coverage-area computation is reserved for evaluation. These targets are *not* produced by a policy-independent classical solver: they come from a best-improvement editor (add/remove/swap) initialized at the policy's own greedy-decoded seed and scored on discretized visibility. We refer to this policy-seeded refinement as "LS" throughout; it is the same edit family whose *learned* counterpart we analyze in Section 6.4, here run to its best-improvement optimum as an oracle teacher rather than learned.

Sizes: the model dimensions of Section 4.2 take the following values:

working width d	128
self-attention blocks L	3
attention heads H	8
FFN expansion factor f	2
policy parameters	$\approx$364k
SetPredictor parameters	464,001

Optimization: weighted BCE (Eq. (10)) with positive-class weight $w_+ \approx 5.66$ (the empirical non-guard-to-guard ratio on `train`), 60 epochs, batch size 32, learning rate 3×10^{-4}, AdamW [36]. Every SetPredictor variant — the

headline probe, the no-encoder ablation, and the capacity-ladder rungs (Section 6.2) — is four independent runs with probe seeds $\{1234, 11, 22, 33\}$ (reported as mean ± std); all figures display seed 1234. These four seeds are always SetPredictor *probe* seeds; the underlying PO/BT policy is the single run described above, not re-seeded. Probe checkpoints and the decision threshold are selected on `dev`, whereas `test`, `ood`, and `ood-large` are touched only after training and threshold selection are complete. The headline threshold $t = 0.20$ is the coverage-favoring end of the swept range $\{0.20, 0.25, 0.30\}$: it is the smallest threshold reported and the value at which the probe clears the 0.95 coverage gate on every `dev` polygon, at the cost of the highest guard count in the sweep. We report all three thresholds rather than tune t per split.

5.3. Evaluation protocol and metrics

For each polygon we greedy-decode the policy's EOS-truncated seed and evaluate it with CGAL exact polygon visibility [35]; separately, we run one SetPredictor pass, threshold at t to obtain $S(t)$, and evaluate $S(t)$ with CGAL. Let N be the partition size. We report mean coverage $\frac{1}{N}\sum \mathrm{Cov}_P(S)$, the normalized guard count $|S|/n$, and the approximation ratio $|S|/\mathrm{OPT}$ against the vertex-guard optimum. Because the mean hides the tail, we also report the per-polygon counts $\#\{\mathrm{Cov}_P(S) \geq c\}$ for $c \in \{0.95, 0.99, 0.999, 1\}$ and the worst case $\min_P \mathrm{Cov}_P(S)$. We distinguish two notions throughout. *Exact feasibility* is full coverage, $\mathrm{Cov}_P(S) = 1$, as the problem defines it (Eq. (2)); we reserve the words *feasible* and *feasibility* for this exact sense. Separately, the counts $\#\{\mathrm{Cov}_P(S) \geq c\}$ report a family of *coverage gates*; we single out $c = 0.95$ as the 0.95 *coverage gate*, a near-feasibility reporting threshold, and report the gate-clearing rate $\#\{\mathrm{Cov}_P(S) \geq 0.95\}/N$ with Wilson 95% confidence intervals. Clearing the 0.95 gate does *not* mean a polygon is feasible in the exact sense. These reporting gates are distinct from the training-time coverage gate $\tau = 0.99$ of Eq. (3), which shapes the reward rather than scoring the output. Where we compare the policy's and the probe's gate-clearing rates on the same polygons, we test the paired per-polygon outcomes with an exact McNemar test. As noted in Section 5.1, OPT values are taken from the vertex-guard optimal solutions distributed with the AGPVG library (for library instances) or computed via ILP (for the `random` class). For 79 of the `ood-large` polygons (all with $n \geq 800$) no optimum is available — most are library instances whose distributed solutions do not cover them (they are open at the source), and for the remainder our ILP did not terminate within budget. Restricting $|S|/\mathrm{OPT}$ to the 206 solved instances would bias the ratio toward the easier 600–700-vertex polygons, so on `ood-large` we report coverage and the normalized guard count $|S|/n$ over all 285 polygons but no approximation ratio.

Computational cost. All experiments were run on a single workstation with an AMD Ryzen 9 3900X CPU, 64 GB RAM, and an NVIDIA GeForce RTX 2080 SUPER GPU (8 GB VRAM). Training the SetPredictor probe is light: about 5 s per epoch, ≈ 5 minutes for a 60-epoch run on the GPU above (the four-seed

ensemble, $\approx$ 20 minutes); the frozen PO/BT policy is reused rather than retrained. The geometric preprocessing — the exact per-vertex CGAL visibility polygons and the $M = 500$ discretized-visibility matrix — is computed once per polygon and cached for the $\sim$9.9k library polygons, so it is not repeated across rollouts, edits, or seeds. At inference the method is a single geo-free forward pass, tens of milliseconds on the GPU above; Section 6.5 times it in full and sets it against the classical visibility-based pipeline. Exact CGAL coverage is used solely for evaluation and is the dominant offline cost, scaling with the guard-set size.

6. Results

The results follow the hypothesis: the policy places guards but leaves a tail (Section 6.1), reading the frozen encoder closes it (Section 6.2), the effect generalizes out of distribution (Section 6.3), and a single pass suffices because the probe is a fixed point under iteration (Section 6.4).

6.1. The policy places guards

The geo-free policy on its own (Table 3, *seed* row) places guards at near-classical cardinality — its mean $|S|/n$ (0.166) is close to classical greedy (0.152), at $|S|/\mathrm{OPT} \approx 1.09$ — and clears the 0.95 coverage gate on 293 of the 362 `test` polygons (81%, Wilson 95% CI: 0.766, 0.847). The LS oracle (bottom row) clears the 0.95 gate on all 362 at lower cardinality ($|S|/n = 0.137$, $|S|/\mathrm{OPT} = 0.885$), but is not geo-free: it reads discretized visibility at every add/remove/swap step and serves here only as the probe's training target, not as a geo-free competitor. The results show that vertex-guard placement is learnable, to a degree, by a method that never reads an explicit visibility object at inference. The remaining 69 polygons (19%) fall below the 0.95 gate under the policy alone; this tail is the paper's central motivating observation (the distribution is shown in Section 6.2), and the method gives no coverage guarantee on its own. Figure 2 shows the seed, the seed plus probe at $t = 0.20$, and the optimum side by side for an in-distribution and an out-of-distribution polygon.

6.2. Reading the encoder closes the tail

The policy's mean coverage hides a bimodal distribution (Table 4): it reaches full coverage on only a handful of polygons and trails a long left tail down to a worst polygon at 0.830, while the SetPredictor at $t = 0.20$ eliminates that tail and lifts the bulk above 0.99. On the displayed seed the probe clears all 69 of the policy's below-gate polygons and introduces no new failures ($69 \to 0$ below 0.95; paired exact McNemar test, $p \approx 3 \times 10^{-21}$); the same paired test is applied at OOD scale in Section 6.3. The shift also holds under the strict full-coverage requirement that defines feasibility ($\mathrm{Cov}_P(S) = 1$): the probe covers 240 of the 362 polygons exactly against the policy's 3 (Table 4), so the gain is not an artifact of reading only the loose 0.95 gate.

Method	Mean cov	$\#\{\text{Cov} \geq 0.95\}/N$	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert/\text{OPT}$
Classical baseline (non-learned)				
Greedy	0.9994	362/362	0.1524	1.0092
Learned policy / probe				
Pretrained pointer (seed)	0.9689	293/362 (0.766,0.847)	0.1664	1.0885
SetPredictor full ($t = 0.20$)	**0.9985** $\pm$ 0.0010	361.5 $\pm$ 0.9/362	0.3888 $\pm$ 0.0603	2.5560 $\pm$ 0.4027
SetPredictor, no-encoder ($t = 0.20$)	0.9972 $\pm$ 0.0013	361.8 $\pm$ 0.4/362	0.4781 $\pm$ 0.0504	3.1093 $\pm$ 0.3350
*Oracle editor (*SetPredictor *target, policy-seeded)*				
LS on policy seed	0.9948	362/362	0.1369	0.8854†

Table 3: Held-out evaluation on `test` (362 polygons). Classical greedy is the only non-learned anchor. The *seed* row is the PO/BT policy decoded greedily; *LS on policy seed* is its local-search refinement — the SetPredictor's supervised target, not a classical baseline (Section 5.2). SetPredictor *full* and *no-encoder* are the headline probe and its zeroed-embedding ablation, each mean $\pm$ std over four seeds $\{1234, 11, 22, 33\}$ at $t = 0.20$ (Section 6.2). Both probes saturate the 0.95 gate ($\geq 361/362$); the encoder's effect surfaces at the stricter gates of Table 4 and in the $\approx$20% higher guard cost the no-encoder probe pays ($\lvert S\rvert/\text{OPT}$ 3.11 versus 2.56). Wilson 95% CI shown only where the proportion is informative.
$^\dagger \lvert S\rvert/\text{OPT} < 1$ for the oracle editor because it halts at a disc-vis coverage gate ($\tau = 0.99$) and slightly undercovers (mean exact coverage 0.995), so it uses fewer guards than the *full-coverage* optimum. This is an artifact of the training reward, not a loose OPT; the $\lvert S\rvert/n$ column is free of this caveat.

Method	$\#\{\text{Cov} = 1\}$	$\#\{\text{Cov} \geq 0.999\}$	$\#\{\text{Cov} \geq 0.99\}$	$\#\{\text{Cov} \geq 0.95\}$	min Cov
Pretrained pointer (seed)	3	15	97	293/362	0.830
SetPredictor $t = 0.20$ (full)	240	310	358	362/362	0.977
SetPredictor $t = 0.20$, no-encoder	120	163	306	362/362	0.952
SetPredictor $t = 0.25$ (full)	194	280	356	362/362	0.962
SetPredictor $t = 0.30$ (full)	151	219	350	362/362	0.952

Table 4: Per-polygon coverage distribution on `test` (362 polygons). Counts are polygons reaching each coverage threshold; *min* $\text{Cov}_P(S)$ is the worst polygon. The policy has a long left tail; the SetPredictor shifts the whole distribution right. This table exposes the encoder's contribution: where the 0.95 gate saturates, the stricter 0.99/0.999 gates and $\min \text{Cov}_P(S)$ separate the full probe from the no-encoder ablation — at $t = 0.20$, 4 polygons below 0.99 versus 56, and a worst polygon lifted from 0.952 to 0.977. All rows are the displayed seed (1234); Table 3 gives the four-seed mean$\pm$std.

The next question we ask is whether closing that tail is the encoder's doing or coordinates alone suffice. We isolate the encoder's contribution by retraining the probe with the frozen embedding h_i masked to zero on every forward pass — architecture, parameter count, optimizer, and training schedule identical (Section 5.2). The 0.95 coverage gate does not reveal the encoder's contribution, as both probes saturate there (362/362, Table 3) and the headline mean coverage is likewise nearly identical. The encoder's contribution lives in the tail. At the stricter 0.99 gate of Table 4, on the displayed seed the full probe leaves 4 polygons below against 56 for the no-encoder ablation, a fourteen-fold difference, and lifts the worst polygon from 0.952 to 0.977 (across the four seeds the mean count below 0.99 is ≈ 11 for the full probe and ≈ 29 for the no-encoder, of which the displayed seed's 56 is the worst — Table 5). The gap becomes qualitative out of distribution (Section 6.3). This is the direct measurement behind C1 — the frozen encoder carries geometric structure that raw coordinates do not reproduce, even with the decoder's full parameter budget.

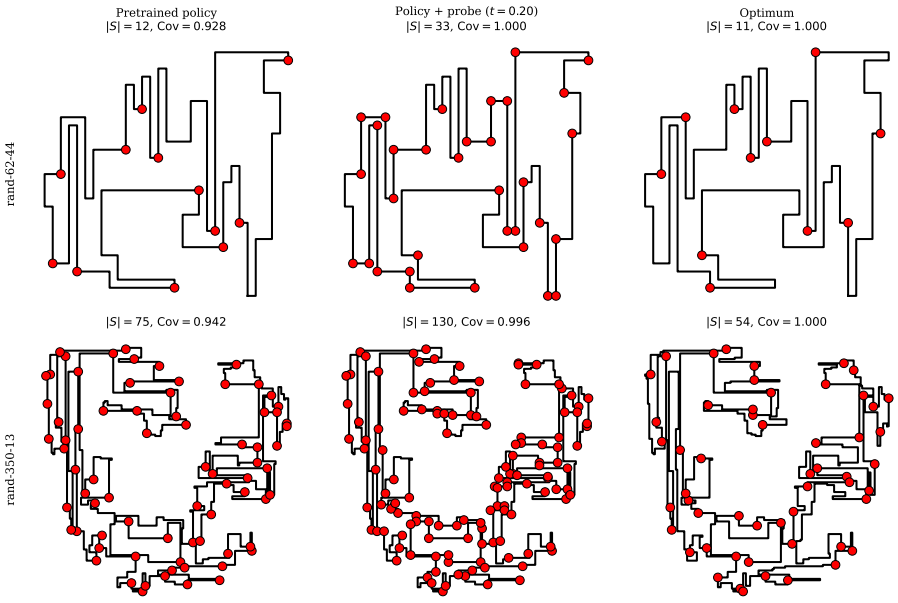

Figure 2: Two polygons drawn with the policy's seed (left column), the policy plus the SET-PREDICTOR probe at $t = 0.20$ (middle column), and the optimum (right column). Vertices selected as guards are marked. Row 1 is an in-distribution polygon where the policy already covers most of the region. Row 2 is an out-of-distribution polygon ($n = 350$, about $1.8\times$ the largest training polygon) where the policy's seed falls below the 0.95 coverage gate ($\mathrm{Cov}_P(\cdot) = 0.942$) and the probe lifts it back above ($\mathrm{Cov}_P(\cdot) = 0.996$) at a guard cost typical for this split ($|S|/\mathrm{OPT} \approx 2.4$; cf. Table 9).

The no-encoder ablation removes the encoder but keeps the policy's seed indicator among the probe's inputs, so the recovered coverage could in principle be credited to the decoder's guard set rather than to the encoder. We therefore complete the 2×2 design, masking the seed indicator as well (Table 5, four-seed mean $\pm$ std at $t = 0.20$). The two axes must be read together, because at a fixed threshold the four conditions trade coverage against guard count differently. Masking the seed alone barely moves the probe: the encoder-only probe nearly matches the full probe both in the tail (17.2 ± 2.4 versus 11.0 ± 8.7 polygons below 0.99) and in cost ($|S|/\mathrm{OPT}$ 2.42 versus 2.56). Masking the encoder alone is markedly worse on the tail (29.0 ± 18.0 below 0.99 at $|S|/\mathrm{OPT}$ 3.11). The coords-only probe (neither input) appears to have the smallest tail of all, but this is misleading: its threshold barely functions — it selects 0.72, 0.70, and 0.67 of all vertices at $t = 0.20, 0.25, 0.30$ — so it holds coverage by *flooding* guards rather than selecting them, never entering the guard-count regime the other probes operate in ($|S|/\mathrm{OPT} = 4.64$, against 2.4–2.6). Reading the axes together, the seed contributes little, whereas the encoder is what makes economical coverage reachable at all: it is the representation, not the decoder's seed, that lets the probe place few guards and still cover.

Probe inputs	$\#\{\text{Cov} < 0.99\}$	min Cov	$\lvert S\rvert/n$	$\lvert S\rvert/\text{OPT}$
SetPredictor full	11.0 ± 8.7	0.951 ± 0.029	0.39 ± 0.06	2.56 ± 0.40
no-seed (encoder only)	17.2 ± 2.4	0.918 ± 0.029	0.37 ± 0.01	2.42 ± 0.07
no-encoder (seed only)	29.0 ± 18.0	0.947 ± 0.022	0.48 ± 0.05	3.11 ± 0.34
coords-only (neither)	0.8 ± 0.8	0.962 ± 0.044	0.72 ± 0.01	4.64 ± 0.06

Table 5: Encoder–seed ablation on `test` (362 polygons), all probes at $t = 0.20$, four-seed mean $\pm$ std over $\{1234, 11, 22, 33\}$. The four rows are the 2×2 on/off combinations of the two probe inputs — the frozen encoder embedding and the policy's seed indicator; an absent input is masked to zeros, leaving architecture and parameter count unchanged. The two axes must be read together — the coverage tail ($\#\{\text{Cov} < 0.99\}$, min Cov) against guard cost ($|S|/n$, $|S|/\text{OPT}$) — because the same threshold $t = 0.20$ does not produce the same operating regime across rows: a probe that selects nearly all vertices will show a short tail simply by over-guarding, not by genuinely learning to place guards. The encoder-only probe nearly matches the full probe on both; the seed-only probe (the no-encoder row of Table 4) widens the tail at higher cost. The coords-only probe is *degenerate*: its threshold barely functions — it selects 0.72, 0.70, 0.67 of all vertices at $t = 0.20, 0.25, 0.30$ — so it holds coverage by *flooding* guards rather than selecting them, and its small $\#\{\text{Cov} < 0.99\}$ is an artifact of over-guarding ($|S|/\text{OPT} = 4.64$), not selection quality. Only the encoder-bearing probes reach the economical regime ($|S|/\text{OPT} \approx 2.4$–$2.6$); without the encoder the probe is stuck at 3.1× (seed only) or 4.6× (neither). The encoder is what makes economical coverage *reachable*.

The four conditions calibrate differently, so comparing them at one threshold can mislead: a probe that selects most vertices shows a short tail for free. Table 6 therefore reports every condition across the swept curve $t \in \{0.20, 0.25, 0.30\}$, which lets us compare them at a matched guard budget $|S|/n$ rather than a matched threshold. The ranking holds. At a matched budget $|S|/n \approx 0.31$ (full and no-seed at $t = 0.30$) the encoder-only probe trails the full probe by little, 38 versus 31 polygons below 0.99, so the seed contributes little. The no-encoder probe never matches the full probe's tail at a comparable budget: its shortest tail, 29 below 0.99, already uses $|S|/n = 0.48$, more than the full probe's 0.39 at $t = 0.20$ that leaves only 11; lowering its threshold to save guards collapses coverage instead (152 below 0.99 at $|S|/n = 0.26$). The coords-only probe stays in the flooding regime at every threshold ($|S|/n \geq 0.67$), so its short tail is paid for entirely in guards. This coords-only condition answers a natural baseline question: it is the same 464K Transformer trained directly on coordinates against the same LS targets, with the 128 encoder channels masked to zero. It is not competitive, and holds coverage only by flooding guards. The encoder's representation, not the coordinates or the Transformer, is what makes an economical budget reachable.

Since the SetPredictor is itself a small Transformer, one might worry that *it* computes the geometry and the encoder merely passes coordinates through. To rule this out we replace the probe with a plain linear classifier — no hidden layers, no attention — that reads only the frozen per-vertex features and separates guard from non-guard vertices. It already reaches ROC-AUC ≈ 0.84, a measure of how reliably the rule ranks a guard above a non-guard (0.5 is chance,

Probe inputs	t	$\lvert S\rvert/n$	$\lvert S\rvert$/OPT	#{Cov < 0.99}	min Cov
SetPredictor full	0.20	0.39	2.56	11.0	0.951
	0.25	0.34	2.26	18.5	0.939
	0.30	0.31	2.02	31.2	0.934
no-seed (encoder only)	0.20	0.37	2.42	17.2	0.918
	0.25	0.33	2.18	26.2	0.907
	0.30	0.30	1.98	38.2	0.887
no-encoder (seed only)	0.20	0.48	3.11	29.0	0.947
	0.25	0.35	2.29	80.2	0.915
	0.30	0.26	1.71	152.0	0.887
coords-only (neither)	0.20	0.72	4.64	0.8	0.962
	0.25	0.70	4.48	1.5	0.952
	0.30	0.67	4.31	4.5	0.933

Table 6: Encoder–seed ablation across the operating curve on `test` (362 polygons), four-seed mean over $\{1234, 11, 22, 33\}$ at $t \in \{0.20, 0.25, 0.30\}$ — the matched-*budget* companion to Table 5. Compare rows at a matched $|S|/n$ rather than a matched threshold; the reading is in the text.

1.0 perfect).[2] A linear readout cannot be credited with computing geometry itself [10, 11], so the guard-relevant structure must already be linearly accessible in the frozen embedding — the encoder, not the probe, carries it. Scored by such a linear rule, guard vertices pile up at higher scores than non-guards, with only modest overlap, as visible in Figure 3.

Probe capacity. The SetPredictor is an expressive model, so we should separate what the representation contains from what the probe computes on top of it [11, 13]. A capacity ladder over the same frozen features and held-out protocol does this (Table 7): a linear readout, an attention-free MLP (the SetPredictor with its self-attention blocks removed, leaving a per-vertex projection with masked-mean-pooled seed and global context), a single self-attention layer, and the full three-layer probe. Per-vertex ROC-AUC rises with capacity but saturates early: 0.837 (linear), 0.909 (MLP), 0.912 (one layer), 0.923 (full). The attention-free MLP recovers about 84% of the linear-to-full gain; the two attention rungs add only the remaining ≈ 0.014. A modest nonlinear readout of the frozen embedding is thus enough, and the full Transformer's capacity adds little. The guard-relevant geometry sits in the encoder; the probe does not build it. We compare capacities on ROC-AUC, which is independent of the threshold, because a fixed threshold puts different-capacity probes at different operating points (the calibration caveat of Table 6).

Closing the tail has a price, with the threshold t as the knob: the approximation ratio rises as t falls (Table 8), reaching $\approx 2.6\times$ optimum at the headline threshold against ≈ 1.1 for the policy alone. Figure 4 gives the full picture in

[2] L_2-regularized logistic regression under 5-fold polygon-grouped cross-validation over 200 polygons / 12,424 vertices: ROC-AUC $= 0.843 \pm 0.009$ and PR-AUC $= 0.582 \pm 0.027$ (mean $\pm$ std across folds) against a 0.16 positive-class prior.

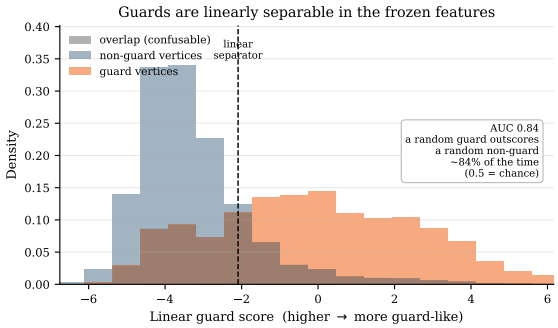

Figure 3: Guard vertices are linearly separable in the frozen encoder features. A one-dimensional linear rule (LDA) over the same 200-polygon / 12,424-vertex probe set scores each vertex; the histograms show that score for guard and non-guard vertices separately. The rule is fit and scored *out-of-fold* (GroupKFold over polygons — no vertex is scored by a rule that saw its own polygon), so the separation reflects embedding structure rather than memorization. Guards score higher; the shaded band is the overlap the rule confuses. The separation corresponds to ROC-AUC ≈ 0.84, matching the logistic probe in the text.

Probe capacity	Params	ROC-AUC	PR-AUC
Linear (logistic)	129	0.837 ± 0.000	0.531 ± 0.000
Attention-free MLP	66,561	0.909 ± 0.001	0.633 ± 0.003
1 self-attention layer	199,041	0.912 ± 0.003	0.643 ± 0.011
Full SetPredictor (3 layers)	464,001	0.923 ± 0.002	0.658 ± 0.015

Table 7: Probe-capacity ladder: held-out per-vertex ROC-AUC / PR-AUC on the 362-polygon `test` split against the LS target. Attention-bearing rungs are four-seed mean ± std; the linear rung is a single held-out logistic fit (consistent with the cross-validated 0.843 in the text). Most of the linear-to-full gain arrives with no self-attention at all.

and out of distribution: the coverage CDF sits to the right of the policy at every threshold, while the $|S|/n$ and $|S|/\mathrm{OPT}$ box plots show the matching cost. We report the trade-off rather than fix one operating point.

6.3. Out-of-distribution generalization

The next test of the representation is the `ood` split, a larger held-out set that spans the training size range and extends well beyond it, reaching $n = 1000$ — about five times the largest training polygon. Table 9 reports it. The policy alone drops to mean coverage 0.957, with 303/2081 polygons below the 0.95 gate (85% clear it). The probe at $t = 0.20$ (four-seed mean) cuts that to 16.8 ± 12.2 (99% clear the gate) at mean coverage 0.995 — roughly an order-of-magnitude reduction in failures. The improvement is significant for every probe seed individually (per-seed failure counts 3, 8, 22, 34; exact McNemar test on the paired per-polygon outcomes, $p < 1.2 \times 10^{-75}$ in all four cases), and the per-seed Wilson 95% intervals for the probe's gate-clearing rate all lie above 0.977. The no-encoder ablation leaves 44 ± 23 below the gate (four-seed mean) at mean

Threshold t	Mean Cov	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert$/OPT	$\#\{\text{Cov} \geq 0.95\}/N$
0.20	0.9993	0.4376	2.8972	362/362
0.25	0.9988	0.3802	2.5082	362/362
0.30	0.9979	0.3353	2.2041	362/362

Table 8: Cost of coverage on `test` (362 polygons), displayed seed (1234). The four-seed mean $\lvert S\rvert$/OPT is 2.02 ± 0.21 at $t = 0.30$ and 2.56 ± 0.40 at $t = 0.20$.

Method	Mean cov	$\#\{\text{Cov} \geq 0.95\}/N$	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert$/OPT
Pretrained pointer (seed)	0.9567	1778/2081 (0.839,0.869)	0.1807	1.1488
SetPredictor full ($t = 0.20$)	$\mathbf{0.9946} \pm 0.0020$	$2064.2 \pm 12.2/2081$	0.3617 ± 0.0447	2.3341 ± 0.2951
SetPredictor, no-encoder ($t = 0.20$)	0.9849 ± 0.0055	$2037.2 \pm 22.6/2081$	0.3667 ± 0.0550	2.3583 ± 0.3646

Table 9: Out-of-distribution evaluation on `ood` (2081 polygons, n up to 1000). Rows as in Table 3: the policy seed, the full probe, and the no-encoder ablation, both probes the four-seed mean $\pm$ std at $t = 0.20$.

coverage 0.985 — about 2.6× the full probe's failures (Table 9), so out of distribution the encoder's contribution is qualitative, not the saturated tie it is in distribution. This is the evidence behind C2: a small classifier over the frozen embeddings recovers gate coverage across this held-out set. The gain is not an artifact of the in-distribution-size majority of `ood`: on the genuinely larger-than-training polygons alone ($n > 198$, 885 instances) the policy clears the gate on 85% and the probe on 98.7% across four seeds (100% on the displayed seed). The reduction is robust across seeds, though the residual failure count is seed-dependent (the per-seed counts above), so at-scale generalization is partially seed-dependent. Figure 4 (middle and bottom rows) shows the matching coverage and cost distributions for `ood` and `ood-large`.

Extreme out-of-distribution. On the `ood-large` split (n from 600 to 2250, up to about eleven times the largest training polygon), the worst-case coverage is the more informative metric, as the mean can look reasonable while the tail collapses (Table 10). The pretrained seed leaves its worst polygon at coverage 0.038 with 75/285 below the 0.95 gate, and the no-encoder ablation barely moves it: its worst-polygon coverage averages only 0.09 ± 0.09 across the four seeds (three of the four still pinned at 0.038), so it trails the full probe on the mean (0.930 versus 0.963) and collapses on the tail. The full probe lifts the worst polygon for every seed: across the four seeds the worst-polygon coverage at $t = 0.20$ averages 0.61 ± 0.29 (against the no-encoder's 0.09 ± 0.09), and on the displayed seed reaches 0.951 with nothing left below the gate. The seed dependence is genuine — the four-seed gate-clearing count is 253.5 ± 19.9 of 285, with the per-seed count below the gate ranging from 0 (the displayed seed) to 55. At $t = 0.20$ the probe uses $\lvert S\rvert/n \approx 0.30$ guards on this split, the cost of restoring coverage this far beyond the training range.

6.4. Alternatives to the probe: decode search, editors, and the fixed point

Decode-time search does not close the tail. The simplest alternative to the probe is to decode the policy better rather than read its encoder. We test this directly

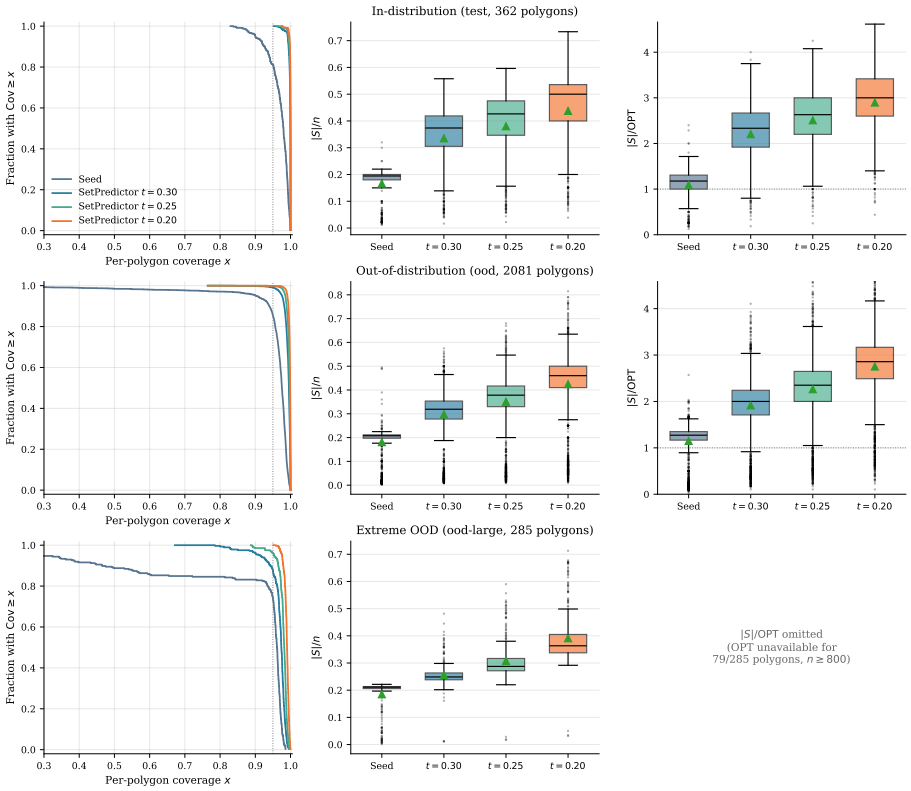

Figure 4: Distributional view across all three regimes: `test` (top, 362 polygons), `ood` (middle, 2081), and `ood-large` (bottom, 285, n up to 2250). *Left:* per-polygon coverage as a complementary stair-step CDF; the dashed line marks the 0.95 coverage gate. *Center:* box plots of $|S|/n$ for the policy seed and the three probe thresholds. *Right:* the approximation ratio $|S|/\mathrm{OPT}$ for `test` and `ood` only; omitted for `ood-large`, where OPT is unavailable for the larger polygons (Section 5.3). Boxes show the displayed seed (1234); four-seed mean±std is in Tables 3 and 9.

(Table 11): for each `test` polygon we draw $K = 32$ stochastic rollouts from the frozen policy and select among them geo-free, by length-normalized log-likelihood, with no visibility oracle. The result is statistically indistinguishable from greedy — 294 versus 293 of 362 clear the 0.95 gate, 266 versus 265 below 0.99, identical $|S|/\mathrm{OPT} \approx 1.09$ — because the policy's maximum-likelihood decode already *is* the greedy seed: likelihood-ranked search never prefers a more-covering set, and occasionally prefers a less-covering one (its worst polygon drops to 0.557). Letting an *oracle* pick the highest-coverage sample of the 32 — classical active search, no longer geo-free — raises gate-clearing to 333/362 but still leaves 199 below 0.99, far short of the probe (all 362 clear the gate, ≈ 4 below 0.99 on the displayed seed). The tail is thus not a decoding artifact that more search removes: closing it requires reading the encoder, and the probe

Method	Mean Cov	min Cov	$\#\{\text{Cov} \geq 0.95\}/N$	Mean $\lvert S\rvert/n$
Pretrained pointer (seed)	0.8678	0.038	210/285	0.1849
SetPredictor full ($t = 0.20$)	$\mathbf{0.9630} \pm 0.0209$	0.612 ± 0.286	$253.5 \pm 19.9/285$	0.2951 ± 0.0579
SetPredictor, no-encoder ($t = 0.20$)	0.9295 ± 0.0196	0.091 ± 0.092	$251.0 \pm 13.8/285$	0.3053 ± 0.0477

Table 10: Extreme out-of-distribution evaluation on `ood-large` (285 polygons, n from 600 to 2250), coverage scored by exact CGAL at $t = 0.20$ over all 285. The approximation ratio is omitted because OPT is unavailable for 79 polygons ($n \geq 800$; Section 5.3). The full-probe and no-encoder rows are four-seed mean ± std; their min Cov is the mean ± std of the four per-seed worst-polygon coverages.

Decode of the frozen policy	$\#\{\text{Cov} \geq 0.95\}/N$	$\#\{\text{Cov} < 0.99\}$	Mean Cov	min Cov	$\lvert S\rvert/n$	$\lvert S\rvert/\text{OPT}$
greedy (seed)	293/362	265	0.969	0.830	0.166	1.09
best-of-32, likelihood (geo-free)	294/362	266	0.967	0.557	0.166	1.09
best-of-32, coverage oracle (*not* geo-free)	333/362	199	0.981	0.845	0.171	1.12

Table 11: Decode-time search on `test` (362 polygons), all reading the same frozen PO/BT policy. *greedy* is the seed of Table 3. *best-of-*32*, likelihood* draws $K = 32$ stochastic rollouts and keeps the one with the highest length-normalized log-likelihood — a geo-free decode search with no visibility oracle. *best-of-*32*, coverage oracle* instead keeps the highest-coverage sample (tie-break: fewer guards); it consults exact coverage to choose and is therefore *not* geo-free, reported only as an upper bound on what the policy's own samples contain. Greedy is included in each candidate pool, so neither selection can underperform it on its own ranking metric. Geo-free search matches greedy and does not close the tail; even oracle-assisted search falls well short of the SetPredictor (Table 4).

pays for that in guards ($|S|/\text{OPT} \approx 2.6$) rather than in search.

Before settling on the single-shot probe we tried to improve the seed with a learned *iterative* editor applying ADD/REMOVE/STOP edits. Three architectures of increasing capacity — including one with topology features and an auxiliary visibility-prediction loss, and one with DAgger refresh [32] — failed to improve over the seed in any coverage-preserving regime. The best variant cut $|S|$ by about 24% but lost coverage, reaching only a 0.27 *recovery rate* (the fraction of its edits that match or beat the local-search reference on both coverage and guard count) — well below replacement quality. Three structural reasons recur: errors accumulate because the editor sees clean trajectories in training but its own predictions at inference; a single $\{\text{ADD}, \text{REMOVE}, \text{STOP}\} \times \{1, \ldots, n\}$ head couples selection with termination, miscalibrating STOP; and the train/inference state distributions diverge in ways DAgger refresh did not close.

The same teacher-forcing cascade also defeats a *supervised* pointer network — which we trained directly on AGPVG solutions before adopting the reinforcement recipe. Cloning a guard sequence is ill-defined for a second reason beyond the cascade: a polygon has many optimal and near-optimal guard sets (the constraint $\text{Cov}_P(S) = 1$ in Eq. (2) fixes the cardinality, not the membership), so any single target sequence is an arbitrary choice among equivalent solutions, and an autoregressive loss penalizes the model for proposing a different but equally valid one. Forced onto sequences it would not itself generate, the recurrent decoder then drifts into hidden states unseen during training and collapses (mean coverage ≈ 0.50 at roughly $3\times$ the optimum) — the brittleness of supervised cloning anticipated in Section 1. Preference optimization sidesteps both

problems: it ranks the policy's *own* rollouts by reward rather than matching a designated target, so non-unique optima are not in conflict. The single-shot probe likewise has none of these failure modes — no rollout, no stop token, and a per-vertex membership label rather than an ordered sequence, trained and used in the same regime.

A clarification this invites: the probe's supervised targets $S^{\star}_{\mathrm{LS}}$ are themselves the output of this same edit family run to its best-improvement optimum — an *oracle* editor seeded by the policy. So the contrast is sharper than "editors fail, the probe works": the *learned* editor fails to reproduce the oracle's edits autoregressively, whereas a single-shot probe trained on the oracle's endpoints recovers them in one pass. That the probe succeeds is therefore not, by itself, evidence about the encoder — it could merely be distilling the oracle. What attributes the success to the *representation* is the encoder–seed ablation of Section 6.2 (Table 5), run with the LS target held fixed: masking the seed indicator barely moves the probe, masking the frozen embedding markedly widens the tail, and removing both leaves a probe that stays above the gate only by flooding guards ($|S|/\mathrm{OPT} \approx 4.6$); a zero-capacity linear probe over the same embedding already separates guards (ROC-AUC 0.843). Were the probe merely distilling the oracle, which inputs it reads would not matter — yet it is the encoder, not the decoder's seed, that makes the targets predictable from coordinates alone.

Re-running the probe with its own output as the updated seed indicator does not change the prediction in any meaningful way: across $K \in \{1, 2, 3, 5\}$ the three metrics barely move (Figure 5). Almost all of the movement is a single settle between $K = 1$ and $K = 2$ at the highest threshold ($t = 0.8$) — at most ≈ 0.02 in $|S|/\mathrm{OPT}$ (≈ 0.011 in coverage, ≈ 0.004 in $|S|/n$); beyond $K = 2$ nothing changes by more than $\approx 8.5 \times 10^{-3}$ (in $|S|/\mathrm{OPT}$; coverage and $|S|/n$ stay under 2.5×10^{-3}). We test the fixed point at this higher range $t \in \{0.5, \ldots, 0.8\}$ rather than at the headline $t = 0.20$ by design: a higher threshold selects fewer vertices, so the probe's output set is small and differs most from the large seed set it started from, which makes re-inserting it as the next pass's seed the *most* likely to move the prediction — the stringent test, nearest the decision boundary. At the headline $t = 0.20$ the selected set is large and changes even less under iteration, so the single-pass conclusion is only more firmly supported there. This is the fixed-point property (C4): with the seed indicator already among its inputs, a single pass suffices.

6.5. Runtime and memory: geo-free versus classical

Geo-free inference moves the visibility cost from inference to training; Table 12 shows the effect on real polygons across the size grid used for the learned-pipeline timing. At inference the learned pipeline is one coordinate-only forward pass, a policy encode plus a probe pass, costing 6–57 ms on a GPU and 12–264 ms on a CPU for n from 200 to 2000 (same workstation as Section 5.3; the probe alone is ≈ 10 ms at $n = 2000$).

Table 12 reports two classical implementations, because the choice that matters for training also matters here. *Exact greedy* is the implementation used for every guard-count and coverage comparison elsewhere in the paper (Table 3

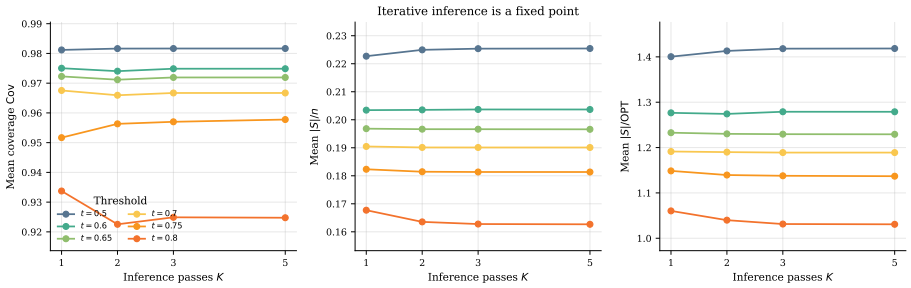

Figure 5: Iterative inference is a fixed point across all three metrics, on the combined `dev` and `test` pool. Each panel shows mean coverage (*left*), $|S|/n$ (*centre*), and $|S|/\mathrm{OPT}$ (*right*) versus inference passes $K \in \{1, 2, 3, 5\}$ for six thresholds $t \in \{0.5, 0.6, 0.65, 0.7, 0.75, 0.8\}$ — a higher range than the headline thresholds, where the fixed-point property is most stringently tested near the decision boundary. The metrics barely move (bounds in Section 6.4); almost all of the change is a single settle between $K = 1$ and $K = 2$ at $t = 0.8$.

onward): each candidate's marginal gain is an exact CGAL polygon-set union. That is the right choice for a quality anchor, but greedy's own selection loop is, like a PO/BT rollout, a many-query-per-polygon computation — roughly $n/6$ rounds, each scanning up to n candidates — and exact scoring is not the fastest a classical solver could be at that. Building the per-vertex visibility polygons it needs before selecting anything already costs 0.12 s at $n = 200$ and 1.6 s at $n = 2000$, 19–34× the learned GPU pass; with selection on top the full run takes ≈ 13.8 s at $n = 200$, and its exact-area union is $O(n^2)$ in polygon operations and exhausted the CGAL backend on larger instances, so we report it only where it is stable.

Disc-vis greedy is the same additive algorithm scored instead on the discretized visibility matrix of Eq. (4) — the same $M = 500$-sample approximation Section 4.1 uses for PO/BT rewards and the local-search targets, applied here to a from-scratch guard-set build rather than seed refinement. Scoring all remaining candidates is one vectorized array operation per round instead of one CGAL call per candidate, and it is close to free: 1.5–78 ms across the grid. The visibility construction it still needs (exact per-vertex visibility plus the M-sample check, a different, non-threaded implementation from the one behind the vis.-precompute column) dominates its cost, 0.34–4.3 s, but unlike exact greedy this is stable at every n we tested, not only $n = 200$. At $n = 200$ it is already $\approx 41\times$ faster than exact greedy's total; it remains 16–28× slower than the learned CPU pass and 57–75× slower than the learned GPU pass across the whole grid — the geo-free advantage holds against the fastest classical alternative we could construct, not only the exact one.

That speed is not a free improvement, and Table 13 shows why. Disc-vis greedy stops the moment its own sample-based estimate crosses the same 0.99 gate used elsewhere, but we score the resulting guard set with exact CGAL after the fact, and the two diverge as n grows: at $n \approx 200$ they agree closely (0.991 estimated against 0.973 ± 0.007 true), but by $n = 2000$ the estimate still

reads 0.990 while true coverage has fallen to 0.807 ± 0.007. The gap tracks guard count more than it tracks n: over a family-diverse sample of 40 polygons spanning $n \in [20, 1750]$, the Spearman correlation of the gap with guard count is 0.95 against 0.88 with n, and controlling for the other separates them (0.78 partial correlation for guard count, 0.29 for n). Two polygons at the same $n = 500$ make the point directly: one needing 3 guards has a gap of +0.004, one needing 56 has a gap of +0.044. We read this as a selection effect rather than plain sampling noise: each round of greedy hill-climbs against the same fixed 500 points, so the more rounds it runs, the more chances it has to land on a guard set that scores well on those particular points without truly covering the polygon. Training does not hit this the same way — PO/BT reward is one noisy sample per rollout, averaged over many rollouts and polygons, and the local-search editor refines an already-good seed with a handful of edits rather than building a guard set from empty over dozens of rounds — which is consistent with why the paper relies on this approximation for training and local-search targets but always re-scores with exact CGAL at evaluation, rather than treating one sample-based estimate as ground truth for a single high-stakes decision.

Memory is easiest to state for the two representations we can bound precisely. The disc-vis matrix is $n \times 500$ booleans, 100 KB at $n = 200$ growing to 1 MB at $n = 2000$; the learned pipeline is a fixed ≈ 0.83M parameters (≈ 3.3 MB at single precision) plus $O(nd)$ activations, independent of n. Peak process memory was comparable across n for both classical implementations, dominated by fixed library overhead; the growing exact polygon-set representation showed up as the instability reported above rather than as a clean memory signal. None of this makes the learned method a better solver — classical greedy wins on guard count regardless of which scoring rule it uses (Section 6.1) — but it is the basis for the settings in Section 1: when many instances must be solved or a geometry backend is costly, paying the visibility cost once at training beats paying it per instance, whether the classical alternative is exact or approximate.

	Classical		Geo-free learned		
n	vis. precompute (ms)	disc-vis greedy (ms)	CPU (ms)	GPU (ms)	GPU speedup
200	120	340	12	6	19×
500	438	938	33	13	34×
1000	671	1684	89	25	27×
2000	1616	4274	264	57	29×

Table 12: Per-instance inference cost. "vis. precompute" is the CGAL per-vertex visibility construction alone; "disc-vis greedy" is the full `greedy_agp_disc.py` pipeline (visibility, $M =$ 500-sample matrix, and selection — strictly more work than vis. precompute, hence higher); the geo-free columns are the policy-encode-plus-probe pass, with GPU speedup taken over vis. precompute. Exact greedy (≈ 13.8 s at $n = 200$, visibility included) is unstable at larger n and is not tabulated.

n	guards	disc-vis coverage (est.)	exact coverage (true)	gap
200	20.5 ± 7.3	0.991	0.973 ± 0.007	0.018 ± 0.006
500	38.2 ± 18.6	0.991	0.950 ± 0.017	0.041 ± 0.015
1000	75.3 ± 9.0	0.990	0.907 ± 0.010	0.083 ± 0.010
2000	139.0 ± 10.4	0.990	0.807 ± 0.007	0.183 ± 0.007

Table 13: Disc-vis greedy's stopping quality, mean ± std over 8/5/3/3 polygons per bucket. "disc-vis coverage" is the sample estimate it stops on (target 0.99); "exact coverage" is the same guard set scored by CGAL afterwards.

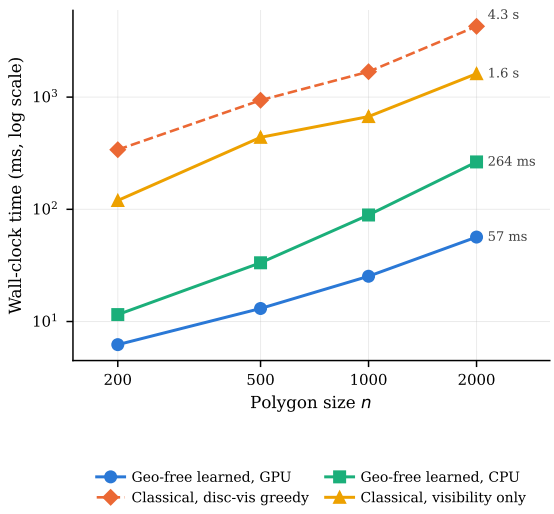

Figure 6: The runtime table as a growth curve (log–log; a constant vertical gap is a constant ratio). Geo-free inference (circles: GPU, squares: CPU) grows slowly with n; classical visibility precompute (triangles) starts almost two orders of magnitude higher; disc-vis greedy (dashed diamonds) is higher still. Exact greedy is omitted here as a single anecdotal point (Table 12: ≈ 13.8 s at $n = 200$, unstable beyond it). Timing data of Table 12; the quality this speed costs is in Table 13.

7. Discussion

The central measurement in this paper is what happens when we read the encoder rather than the decoder. Reading only the frozen embeddings, the coordinates, and the policy's seed indicator, with no visibility matrix or CGAL output, the SetPredictor closes the in-distribution coverage tail on the displayed seed (Table 3) and cuts the out-of-distribution one by about an order of magnitude across four probe seeds (Section 6.3). Two readings fit this. Either the encoder learned enough geometry that a small classifier over its output recovers coverage, or the decoder's EOS calibration was the bottleneck and the policy had already found the right vertices but stopped early. Both give the same headline: the reinforcement method internalized a representation holding more of the geometry than its decoder expressed. The encoder–seed ablation (Table 5) favours the encoder reading, since masking the seed barely changes the probe while masking the encoder widens the tail or forces over-guarding. Three measurements support this (Section 6.2). The no-encoder ablation, masking the embedding with everything else fixed, leaves a probe that cannot match the full probe's coverage at equal cardinality on either split, separating most clearly at the 0.99 gate of Table 4. The no-seed control shows the encoder alone nearly recovers the full probe, so the seed does not carry the signal. And the linear probe (ROC-AUC 0.843 ± 0.009), with the capacity ladder of Table 7 behind it, shows the structure is already accessible from the frozen embedding through a modest readout, not built by the probe. One caveat bounds the claim: successful probing shows the LS target is decodable from the features, which is necessary for the encoder to carry the geometry but does not prove it carries all of it, nor that the residual failures are solely decoder calibration. We give the encoder reading as the better-supported one, not the only one.

Recovering coverage has a price, and we report it rather than disguise it. The probe's guard sets grow to roughly two to three times the optimum as the threshold is lowered (Section 6.2), and the $|S|/\mathrm{OPT}$ box plots of Figure 4 show that the whole distribution widens, not just its mean. A deployment would pick one operating point on this curve; we leave it as a knob because the right point depends on how costly an extra guard is relative to an uncovered region.

Note that we are not claiming a better AGP solver. Classical greedy reaches mean coverage 0.999 at $|S|/n = 0.152$ and $|S|/\mathrm{OPT} = 1.009$ on `test` (Table 3); the probe at $t = 0.20$ reaches mean coverage ≥ 0.998 at $|S|/\mathrm{OPT} \approx 2.6$ across four seeds. A greedy-then-local-search pipeline — the standard high-quality classical recipe — would be stronger still on cardinality; the *LS on policy seed* row of Table 3, which refines a guard set with the same add/remove/swap editor to clear the 0.95 gate on all 362 at $|S|/\mathrm{OPT} = 0.885$, already bounds what such refinement achieves on this data. The classical anchors win on cardinality at preserved coverage, and the policy and probe are not built to contest that. What we document is what is learnable by reinforcement when the model is denied geometric input at the moment of placing guards, and what part of that learning lives in the representation rather than the decoder. The contrast between the failed iterative editors (Section 6.4) and the single-shot probe is the cleanest

expression of the point: removing the stop time and the rollout is exactly what lets the encoder's information surface.

8. Limitations

We acknowledge several limitations of the work, enumerated here for clarity; the first two are conceptual and the rest experimental.

1. **Scope of the "no geometric knowledge" claim.** The geo-free constraint is on inference: the policy and the probe see no visibility object at test time. *Training* does see visibility: the PO/BT reward is computed from discrete visibility coverage, and the probe's supervised targets are generated by local search using discrete visibility as a coverage gate. "No explicit geometric knowledge" must be read as "no geometric oracle at the moment of placing guards."
2. **The probe is supervised, not reinforcement.** The SetPredictor is trained by BCE against local-search-derived labels. It is not itself a reinforcement learner. We position it as a representation probe of the RL-trained encoder, not as an RL contribution.
3. **Held-out partition size.** `test` has 362 polygons. Wilson 95% CIs on the gate-clearing proportion are reported, but the absolute count is modest. The OOD evaluation on `ood` (2081 polygons) provides a much larger second held-out evidence base.
4. **Single instance ecosystem; OOD is mainly size extrapolation.** Although the evaluation spans several polygon families (random orthogonal, random-simple, von Koch, boundary-simple) and a wide size range, nearly all instances originate from one ecosystem — the AGPVG library [18] plus our same-format `random` supplement — so the generator and coordinate format are largely shared between training and evaluation. The out-of-distribution axis we stress is therefore predominantly *size* extrapolation (to $\approx 5\times$ the training maximum on `ood`, $\approx 11\times$ on `ood-large`), not a shift to a genuinely different polygon distribution or instance source. Whether the encoder's geometry transfers across instance ecosystems — procedurally different generators, or real floorplans — is untested and left to future work.
5. **Policy selection and seeds.** The released PO/BT policy is a *single* training run (seed 1234); we did not train PO/BT across multiple random seeds. What was compared during development was a handful of different training *recipes* — REINFORCE baselines (Table 1) and alternative fine-tuning objectives — from which PO/BT was chosen by `train`-set performance. Because that selection used only the `train` split, `test` and `ood` play no role in it and the held-out evaluations are not optimistically biased by the choice; selecting on training reward rather than a held-out criterion is a weaker rule that, if anything, risks favoring an over-fit run, so we make no claim the selected policy is optimal. Policy seed-variance is therefore not characterized: the four seeds $\{1234, 11, 22, 33\}$ reported throughout

are SetPredictor *probe* seeds (Table 3, Section 6.3), and quantifying policy-seed variance would require re-running PO/BT training, which we leave to future work.

6. **LSTM, not Transformer, policy.** We trialed a Transformer encoder–decoder policy of roughly 6× the parameters; it reached a comparable coverage–cost operating point (greedy coverage ≈ 0.97 at $|S|/\mathrm{OPT} \approx 1.15$ versus the LSTM pointer's ≈ 1.21 — sparser but no better on the trade-off), one training run failed to converge, and we found no improvement justifying the added capacity. We read this as overparametrization relative to the scalar RL signal rather than evidence against attention models in general — consistent with the fact that the probe, a small Transformer trained on dense per-vertex labels, does work; a careful attention-model study is future work.
7. **Decode-time search and stronger decoders.** We tested decode-time search of the policy seed by best-of-K stochastic sampling with length-normalized scoring (Section 6.4, Table 11): geo-free likelihood-ranked search matched greedy and did not close the tail, so the residual tail is not a decoding artifact. We did not implement exhaustive beam search — the decoder loop is not exposed for per-step beam expansion, and best-of-K sampling is the stronger search in practice — nor symmetry-exploiting RL such as POMO [17]; these act on the decoder and are orthogonal to the representational question of what the encoder has learned, so we leave them to future work.
8. **Discrete visibility approximation in training.** Both the PO/BT coverage reward and the SetPredictor training targets use discrete visibility sampling at $M = 500$ (Section 4.1). Exact coverage-area computation is used only at evaluation; the per-vertex visibility polygons are computed with CGAL in both training and evaluation. We did not sweep M: the sample average has standard error $O(1/\sqrt{M})$, so a larger M — or stratified sampling near reflex vertices, where the uncovered slivers are hardest to detect — would reduce target noise, but quantifying that sensitivity is left to future work.
9. **Reward-hyperparameter sensitivity.** The reward weights and PO settings ($\lambda = 1.0$, $\rho = 3.0$, $R = 8$ rollouts, $\alpha = 0.05$; Section 5.2) were fixed to the values that produced the released checkpoint rather than swept. The design turns on the *form* of the reward — capping coverage at τ and linearly penalizing the deficit (Section 4.1) — rather than on the precise scalars; a sensitivity study over them would require re-running PO/BT training and is left to future work.
10. **Extreme OOD is strong but seed-dependent.** On the `ood-large` split (Table 10, Section 6.3) the frozen-encoder probe lifts worst-case coverage on every seed while the no-encoder ablation cannot, but the per-seed gate-clearing count is variable, so we read it as evidence of the encoder signal rather than a coverage guarantee at this scale; the probe also pays roughly double the local-search guard count, and the vertex-guard optimum is unavailable for the largest polygons there.

11. **Training-curve reconstruction.** Figure 1 is reconstructed from four saved checkpoints (epochs 110, 114, 160, 200), not from an online metric log; per-epoch coverage was not preserved during the original PO/BT run. The trace shows only late-training dynamics, which is sufficient to confirm that the gradient direction does not collapse but does not characterize early-training behavior.
12. **Invariance and vertex ordering.** The recurrent encoder reads vertices in file order, so the pipeline is invariant to translation and axis-aligned scaling by construction (the input is min–max normalized; Section 5.2) but not, in principle, to rotation, reflection, or vertex re-indexing. We measured the effect on `test` by re-running the frozen policy and probe ($t = 0.20$) on transformed polygons, re-decoding the seed each time: under rotation (45°, 90°, 180°) and mirroring the gate-clearing rate is unchanged (362/362 at the 0.95 gate, mean coverage ≥ 0.998), so in aggregate the learned features tolerate these transforms; a cyclic re-indexing of the vertices is slightly more disruptive (359/362 clear the gate, with one small polygon falling to coverage 0.79), consistent with the order-dependence a recurrent encoder introduces. A permutation- or rotation-equivariant encoder (for instance, attention with relative positional encoding) would remove this dependence by construction and is a natural next step.

9. Conclusion

We asked what a neural policy internalizes about vertex-guard placement when trained from a coverage-aware reward over its own rollouts and denied any geometric oracle at inference, and we answered it by reading the encoder rather than the decoder: a single-shot probe over the frozen embeddings closes the in-distribution coverage tail on the displayed seed and compresses the out-of-distribution tail by roughly an order of magnitude across four probe seeds, which we take as evidence that the reinforcement-trained encoder had captured more of the placement geometry than its decoder emitted as guards. The result is a measurement in a deliberately constrained setting, not a competitor to classical solvers on guard count — the learner is not the best AGP heuristic, but it arrived at a usable representation of vertex-guard placement without ever being shown the geometry it had to reason about at inference. Two extensions follow naturally: a stronger encoder, such as a Transformer trained under the same PO/BT objective, may shorten the residual tail directly and leave less work for the probe; and a per-instance threshold, predicted geo-free from the same embeddings but trained against coverage-selected targets exactly as the probe is trained against the visibility-derived labels $S^{\star}_{\mathrm{LS}}$, would replace the global knob with an adaptive one — keeping visibility a training-time target, never an inference-time input. More broadly, the encoder-probing decomposition need not be specific to vertex guarding: we conjecture it transfers to other geometric set-cover problems whose coverage oracle is costly at inference — terrain guarding or range-limited watchtower placement, for instance — where geo-free

inference stays meaningful precisely because the visibility or coverage oracle one would otherwise consult is expensive to evaluate. Whether a frozen encoder carries the requisite structure in those settings is an open question the present method is designed to ask, and one our single-policy study cannot answer on its own: the conclusions here concern the specific PO/BT pointer policy we trained, and establishing that they hold across architectures and learning rules would require probing more than one solver.

Data and Code Availability

The AGPVG instance library is publicly available [18]. The 313 supplemental `random`-class polygons with their ILP-computed vertex-guard optima, the dataset splits, the trained checkpoints, and all training and evaluation code are available at `https://github.com/dseverdi/AGNet`.

Conflict of Interest

The authors declare no competing interests.

References

[1] D. T. Lee, A. K. Lin, Computational complexity of art gallery problems, IEEE Transactions on Information Theory 32 (2) (1986) 276–282.

[2] J. O'Rourke, Art Gallery Theorems and Algorithms, Oxford University Press, 1987.

[3] A. Elnagar, L. Lulu, An art gallery-based approach to autonomous robot motion planning in global environments, in: 2005 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2005, pp. 2079–2084. `doi:10.1109/IROS.2005.1545170`.

[4] B. S. Bigham, S. Dolatikalan, A. Khastan, Minimum landmarks for robot localization in orthogonal environments, Evol. Intell. 15 (3) (2022) 2235–2238. `doi:10.1007/s12065-021-00616-8`.
URL `https://doi.org/10.1007/s12065-021-00616-8`

[5] M. C. Couto, P. J. de Rezende, C. C. de Souza, An exact algorithm for minimizing vertex guards on art galleries, International Transactions in Operational Research 18 (4) (2011) 425–448.

[6] M. Pan, G. Lin, Y.-W. Luo, B. Zhu, Z. Dai, L. Sun, C. Yuan, Preference optimization for combinatorial optimization problems, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025, arXiv:2505.08735.

[7] R. A. Bradley, M. E. Terry, Rank analysis of incomplete block designs: I. the method of paired comparisons, Biometrika 39 (3–4) (1952) 324–345.

[8] R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Machine Learning 8 (3-4) (1992) 229–256.

[9] I. Bello, H. Pham, Q. V. Le, M. Norouzi, S. Bengio, Neural combinatorial optimization with reinforcement learning, arXiv preprint arXiv:1611.09940 (2016).

[10] G. Alain, Y. Bengio, Understanding intermediate layers using linear classifier probes (2017). arXiv:1610.01644.

[11] J. Hewitt, P. Liang, Designing and interpreting probes with control tasks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 2733–2743.

[12] Y. Belinkov, Probing classifiers: Promises, shortcomings, and advances, Computational Linguistics 48 (1) (2022) 207–219.

[13] Z. Zhang, Y. Ma, Z. Cao, H. C. Lau, Probing neural combinatorial optimization models, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 38, 2025, spotlight; arXiv:2510.22131.

[14] R. Narad, L. Boussioux, M. Wagner, Probing neural TSP representations for prescriptive decision support (2026). arXiv:2602.07216.

[15] O. Vinyals, M. Fortunato, N. Jaitly, Pointer networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 28, 2015.

[16] W. Kool, H. van Hoof, M. Welling, Attention, learn to solve routing problems!, in: International Conference on Learning Representations (ICLR), 2019.

[17] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, S. Min, POMO: Policy optimization with multiple optima for reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020.

[18] M. C. Couto, P. J. de Rezende, C. C. de Souza, Instances for the Art Gallery Problem, http://www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery (2009).

[19] S. K. Ghosh, Approximation algorithms for art gallery problems in polygons, Discrete Applied Mathematics 158 (6) (2010) 718–722.

[20] H. González-Baños, A randomized art-gallery algorithm for sensor placement, Proceedings of the 17th Annual Symposium on Computational Geometry (SoCG) (2001) 232–240.

[21] G. L. Nemhauser, L. A. Wolsey, M. L. Fisher, An analysis of approximations for maximizing submodular set functions—i, Mathematical Programming 14 (1) (1978) 265–294.

[22] B. Ben-Moshe, M. J. Katz, J. S. B. Mitchell, A constant-factor approximation algorithm for optimal 1.5D terrain guarding, SIAM Journal on Computing 36 (6) (2007) 1631–1647.

[23] K. Elbassioni, D. Matijević, J. Mestre, D. Ševerdija, Improved approximations for guarding 1.5-dimensional terrains, CoRR abs/0809.0159v1 (2008).

[24] Y.-H. Liao, P.-C. Chang, H.-H. Li, Reinforcement learning for optimize coverage in art gallery problem using Q-learning based in grid world, IEEE Access 13 (2025) 52711–52724.

[25] M. D. Kaba, M. G. Uzunbas, S. N. Lim, A reinforcement learning approach to the view planning problem (2016). arXiv:1610.06204.

[26] M. Deudon, P. Cournut, A. Lacoste, Y. Adulyasak, L.-M. Rousseau, Learning heuristics for the TSP by policy gradient, in: Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR), Vol. 10848 of Lecture Notes in Computer Science, Springer, 2018, pp. 170–181.

[27] Y. Jin, Y. Ding, X. Pan, K. He, L. Zhao, T. Qin, L. Song, J. Bian, Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem, aAAI 2023 (2023). arXiv:2304.09407.

[28] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d'horizon, European Journal of Operational Research 290 (2) (2021) 405–421. doi:https://doi.org/10.1016/j.ejor.2020.07.063.
URL https://www.sciencedirect.com/science/article/pii/S0377221720306895

[29] I. Mehta, S. Taghipour, S. Saeedi, Pareto frontier approximation network (PA-Net) to solve bi-objective TSP (2022). arXiv:2203.01298.

[30] K. Li, T. Zhang, R. Wang, Deep reinforcement learning for multiobjective optimization, IEEE Transactions on Cybernetics 51 (6) (2021) 3103–3114.

[31] C. Caramanis, D. Fotakis, A. Kalavasis, V. Kontonis, C. Tzamos, Optimizing solution-samplers for combinatorial problems: The landscape of policy-gradient methods, neurIPS 2023 (2023). arXiv:2310.05309.

[32] S. Ross, G. J. Gordon, J. A. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011, pp. 627–635.

[33] Z. Zhang, Y. Ma, Z. Cao, H. C. Lau, Unveiling neural combinatorial optimization model representations through probing, openReview forum:agEy9hliY1; extended version of [13] (2025).

[34] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (8) (1997) 1735–1780.

[35] The CGAL Project, CGAL user and reference manual, `https://doc.cgal.org/5.6/Manual/packages.html` (2024).

[36] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: International Conference on Learning Representations (ICLR), 2019.